

Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей

Кафедра информатики

Дисциплина Архитектура вычислительных систем

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
к курсовому проекту
на тему

**СРАВНЕНИЕ ПРОИЗВОДИТЕЛЬНОСТИ ПРОЦЕССОРОВ INTEL
CORE I5-1235U И INTEL CORE I5-1135G7 НА ОСНОВЕ АЛГОРИТМА
ВЫЧИСЛЕНИЯ ЧИСЛА ПИ С ИСПОЛЬЗОВАНИЕМ МЕТОДА
МОНТЕ-КАРЛО**

БГУИР КП 6-05-0612-02 004 ПЗ

Студент

А. М. Бахмат

Руководитель

А. А. Калиновская

Нормоконтролер

А. А. Калиновская

Минск 2025

СОДЕРЖАНИЕ

Введение.....	6
1 Архитектура вычислительной системы.....	7
1.1 Структура и архитектура вычислительной системы.....	7
1.2 История, версии и достоинства	8
1.3 Обоснование выбора вычислительной системы.....	9
1.4 Анализ выбранной вычислительной системы для написания программы	10
2 Платформа программного обеспечения.....	11
2.1 Структура и архитектура платформы	11
2.2 История, версии и достоинства	14
2.3 Обоснование выбора платформы	14
2.4 Анализ операционной системы для написания программы.....	15
3 Теоретическое обоснование разработки программного продукта.....	17
3.1 Обоснование необходимости разработки.....	17
3.2 Технологии программирования, используемые для решения поставленных задач	18
3.3 Связь архитектуры вычислительной системы с разрабатываемым программным обеспечением.....	19
4 Проектирование функциональных возможностей программы	21
4.1 Общая цель программы	21
4.2 Теоретические сведения о методе Монте-Карло.....	21
4.3 Описание функциональных возможностей.....	23
5 Сравнение производительности процессоров.....	25
5.1 Общая структура программы.....	25
5.2 Описание функциональной схемы программы	27
5.3 Описание блок-схемы алгоритма программы.....	29
5.4 Подготовка к тестированию	30
5.5 Результаты измерений с включенным <i>Hyper-Threading</i>	30
5.6 Результаты измерений с выключенным <i>Hyper-Threading</i>	34
5.7 Анализ полученных результатов.....	38
Заключение	46
Список использованных источников	48
Приложение А (обязательное) Справка о проверке на заимствования	50
Приложение Б (обязательное) Листинг программного кода	51
Приложение В (обязательное) Функциональная схема алгоритма,	

реализующего программное средство	65
Приложение Г (обязательное) Блок схема алгоритма, реализующего программное средство	66
Приложение Д (обязательное) Графики производительности процессоров ..	67
Приложение Е (обязательное) Графический интерфейс пользователя.....	68
Приложение Ж (обязательное) Ведомость курсового проекта	69

ВВЕДЕНИЕ

В современной вычислительной технике производительность центрального процессора является ключевым фактором, определяющим эффективность работы всей системы. С постоянным развитием микроархитектур и переходом на новые технологические процессы актуальной задачей становится сравнительный анализ процессоров разных поколений, позволяющий оценить реальный прирост производительности в конкретных сценариях использования.

В рамках данного курсового проекта проводится сравнительный анализ производительности двух процессоров корпорации *Intel*: более нового *Intel Core i5-1235U* (архитектура *Alder Lake*) и его предшественника *Intel Core i5-1135G7* (архитектура *Tiger Lake*). В качестве метода исследования выбрана реализация алгоритма вычисления числа Π с использованием вероятностного метода Монте-Карло. Данный метод активно использует операции с плавающей точкой, генерацию псевдослучайных чисел и условные переходы, что позволяет комплексно оценить эффективность вычислительных ядер.

Актуальность темы обусловлена необходимостью практической оценки заявленных улучшений новой гибридной архитектуры по сравнению с предыдущим поколением. Метод Монте-Карло широко применяется в финансовом и физическом моделировании, криптографии, поэтому понимание того, как различные процессоры справляются с подобными нагрузками, имеет практическую ценность для специалистов.

Цель работы заключается в проведении сравнительного анализа производительности процессоров *Intel Core i5-1235U* и *Intel Core i5-1135G7* на основе алгоритма вычисления числа Π методом Монте-Карло. Для достижения этой цели в ходе исследования будут решены следующие задачи:

- 1 Изучение технических характеристик и архитектурных особенностей процессоров *Intel Core i5-1235U* и *Intel Core i5-1135G7*.
- 2 Теоретическое исследование метода Монте-Карло для вычисления числа Π и анализ его вычислительной сложности.
- 3 Разработка программы на языке *C++*, реализующей данный алгоритм.
- 4 Проведение серии тестов с фиксацией времени выполнения.
- 5 Анализ полученных данных для оценки эффективности процессоров.

В результате исследования будут получены данные о времени расчета числа Π на каждом из процессоров, на основе которых будут построены графики зависимости времени вычисления от количества итераций. Это позволит сделать выводы об эффективности архитектур двух поколений при выполнении вычислений с высокой степенью параллелизма и недетерминированным характером.

Данный курсовой проект выполнен мной лично, проверен на заимствования, процент оригинальности составляет 99,65% (отчет о проверке на заимствования прилагается).

1 АРХИТЕКТУРА ВЫЧИСЛИТЕЛЬНОЙ СИСТЕМЫ

1.1 Структура и архитектура вычислительной системы

Процессоры *Intel Core i5-1235U* и *Intel Core i5-1135G7* представляют собой два поколения мобильных процессоров корпорации *Intel*, которые ориентированы на один сегмент – высокоэффективные ноутбуки и ультрабуки. Оба процессора предназначены для решения задач, требующих баланса между высокой производительностью и энергоэффективностью, однако пути достижения этого баланса у них различаются.

Основные характеристики выбранных процессоров представлены в таблице 1.1 [1].

Таблица 1.1 – Основные характеристики выбранных процессоров

Характеристики	<i>Intel Core i5-1235U</i>	<i>Intel Core i5-1135G7</i>
Кодовое имя архитектуры	<i>Alder Lake-U</i>	<i>Tiger Lake-UP3</i>
Физические ядра	10	4
Количество потоков	12	8
<i>L1</i> кэш	384 KB	384 KB
<i>L2</i> кэш	5 MB	5 MB
<i>L3</i> кэш	12 MB	8 MB
Базовая частота	1.3 GHz	1.5 GHz
Максимальная частота	4.4 GHz	4.2 GHz
Техпроцесс	7 nm	10 nm
Разрядность	64 bit	64 bit
Встроенная графика	<i>Intel Iris Xe Graphics eligible</i>	<i>Intel Iris Xe Graphics</i>
Тип памяти	DDR4, DDR5	DDR4
Архитектура	x86-64	x86-64
<i>Hyper-Threading</i>	+	+
Теплопакет (<i>TDP</i>)	15 Вт	15 Вт

Intel Core i5-1235U – это процессор для ноутбуков на архитектуре *Alder Lake-U*, в первую очередь рассчитанный на офисные системы. Он имеет 10 ядер (2 производительных ядра + 8 энергоэффективных ядер) и 12 потоков [2]. Процессор обладает 12 МБ кэш-памяти третьего уровня (*L3*) и изготавливается по 7-нм техпроцессу, что позволяет ему обеспечивать высокую производительность при низком энергопотреблении (*TDP* 15 Вт). Базовая тактовая частота составляет 1.3 ГГц, с возможностью увеличения до 4.4 ГГц в режиме *Turbo Boost*. Встроенная графика *Intel Iris Xe Graphics* позволяет комфортно работать с мультимедийным контентом и обеспечивает поддержку современных интерфейсов вывода. Процессор поддерживает технологию

Hyper-Threading, которая позволяет обрабатывать до двух потоков на одно Р-ядро, улучшая многозадачность и общую производительность в многопоточных приложениях.

Intel Core i5-1135G7 также предназначен для использования в ноутбуках и ультрабуках, но базируется на предыдущей архитектуре *Tiger Lake-UP3*. Он включает 4 физических ядра и 8 потоков [3]. Базовая тактовая частота составляет 1.5 ГГц, но может достигать до 4.2 ГГц в режиме *Turbo Boost*. Процессор имеет 8 МБ кэш-памяти *L3* и производится по усовершенствованному 10-нм *SuperFin* техпроцессу. Его архитектура оптимизирована для повышения производительности на такт (*IPC*) и энергоэффективности. Теплопакет составляет 15 Вт (с возможностью настройки производителем устройства), что делает его подходящим для ультрабуков. Встроенная графика *Intel Iris Xe Graphics* обеспечивает значительное по сравнению с предыдущими поколениями улучшение производительности в мультимедийных приложениях. Процессор также поддерживает технологию *Hyper-Threading*, позволяющую каждому ядру обрабатывать до двух потоков одновременно, что повышает эффективность многозадачной работы.

Итак, *Intel Core i5-1235U*, благодаря гибридной архитектуре и увеличенному кэшу *L3*, ориентирован на обеспечение высокой многопоточной производительности и энергоэффективности в новых поколениях ноутбуков. В то время как *Intel Core i5-1135G7*, будучи представителем успешной архитектуры *Tiger Lake*, сочетает энергоэффективность с солидной производительностью для повседневных и мультимедийных задач, оставаясь актуальным решением.

1.2 История, версии и достоинства

Intel Core i5-1235U, представленный в 23 февраля 2022 года [2], является частью 12-го поколения процессоров *Core*, известного под кодовым именем *Alder Lake*. Это поколение ознаменовало революционный переход к гибридной архитектуре, сочетающей высокопроизводительные (*P-cores*) и энергоэффективные (*E-cores*) ядра. Ключевым достоинством данной архитектуры является интеллектуальное распределение нагрузки между разными типами ядер, что оптимизирует баланс между производительностью и энергопотреблением.

Intel Core i5-1135G7, выпущенный в 2 сентября 2020 года [3], относится к 11-му поколению процессоров *Tiger Lake*. Это поколение стало важным этапом, закрепившим переход *Intel* на 10-нм супертехпроцесс и представившим новую графическую архитектуру *Intel Iris Xe*. Несмотря на традиционную однородную структуру, процессор предложил значительный прирост производительности на такт (*IPC*) по сравнению с предшественником (*Ice Lake*). Его главными достоинствами стали существенное улучшение интегрированной графики и высокая энергоэффективность, что сделало его

эталонном для ультрабуков своего времени.

Таким образом, *Intel Core i5-1235U* выделяется передовой гибридной архитектурой, которая обеспечивает исключительную многопоточную производительность и современное управление энергопотреблением. *Intel Core i5-1135G7*, в свою очередь, демонстрирует отточенную однородную архитектуру с превосходной для своего класса интегрированной графикой, оставаясь сбалансированным решением для требовательных мобильных задач.

1.3 Обоснование выбора вычислительной системы

Выбор процессоров *Intel Core i5-1235U* и *Intel Core i5-1135G7* для анализа производительности при вычислении числа Π методом Монте-Карло обосновывается следующими ключевыми факторами:

1 Различные архитектурные подходы – процессоры представляют два принципиально разных подхода к организации вычислений: гибридная архитектура *Alder Lake (i5-1235U)* с разделением на производительные и энергоэффективные ядра против классической однородной архитектуры *Tiger Lake (i5-1135G7)*. Это позволяет исследовать, как разные архитектурные решения влияют на выполнение вероятностных алгоритмов, требующих массовых параллельных вычислений.

2 Оптимизация для многопоточных вычислений – метод Монте-Карло идеально подходит для распараллеливания. *i5-1235U* с 12 потоками и *i5-1135G7* с 8 потоками позволяют исследовать эффективность использования множества потоков при обработке независимых случайных испытаний. Особый интерес представляет анализ того, как гибридная архитектура *i5-1235U* распределяет вычислительную нагрузку между разными типами ядер.

3 Репрезентативность и актуальность – оба процессора широко используются в современных ноутбуках среднего ценового сегмента, что делает результаты исследования практически значимыми для широкого круга пользователей. Сравнение процессоров соседних поколений позволяет оценить реальный прирост производительности при переходе на новую архитектуру.

4 Адекватность метода исследования – вероятностный характер метода Монте-Карло, требующий выполнения большого количества однотипных операций, хорошо подходит для сравнительного анализа процессоров, поскольку позволяет оценить как *raw*-производительность, так и эффективность работы подсистемы памяти и кэша.

Таким образом, выбор данных процессоров для исследования обусловлен их архитектурными различиями, ориентированностью на многопоточные вычисления и репрезентативностью для современного рынка мобильных процессоров. Проведение сравнительного анализа позволит выявить преимущества и недостатки каждой архитектуры при решении задач вероятностного характера.

1.4 Анализ выбранной вычислительной системы для написания программы

Для сравнительного анализа производительности процессоров *Intel Core i5-1235U* и *Intel Core i5-1135G7* были использованы результаты авторитетных бенчмарков: *PassMark CPU Mark*, *GeekBench 5* и *Cinebench R15*. Эти тесты позволяют комплексно оценить вычислительные способности процессоров в различных режимах работы, что особенно важно для прогнозирования их поведения при реализации алгоритма вычисления числа Пи методом Монте-Карло.

PassMark CPU Mark – комплексный бенчмарк, состоящий из 8 различных тестов, включая вычисления с целыми числами и плавающей точкой, проверку расширенных инструкций, сжатие, шифрование и расчеты физики [4]. Согласно результатам тестирования, *Intel Core i5-1235U* демонстрирует значительное преимущество в общей производительности с результатом 12845 баллов против 9455 баллов у *i5-1135G7*, что составляет превосходство в 35,9%. Это указывает на существенный прогресс архитектуры *Alder Lake* в многопоточных задачах.

GeekBench 5 – кроссплатформенный бенчмарк, который моделирует реальные вычислительные задачи [5]. В однопоточном тесте, измеряющем производительность одного ядра, *i5-1235U* показывает результат 1832 балла против 1593 баллов у *i5-1135G7*, что соответствует преимуществу в 15%. В многопоточном режиме, использующем все доступные ядра, *i5-1235U* демонстрирует результат 5647 баллов против 4499 баллов у предшественника, что составляет преимущество в 25,5%. Это свидетельствует о более эффективном распараллеливании задач в процессоре нового поколения.

Cinebench R15 – специализированный бенчмарк для тестирования производительности рендеринга, созданный на основе движка *Cinema 4D* [6]. В однопоточном тесте *i5-1235U* показывает результат 218 баллов против 198 баллов у *i5-1135G7*, что соответствует преимуществу в 9,8%. Данный тест особенно важен для оценки производительности в задачах работы с графикой и сложными математическими вычислениями.

Проведенный анализ позволяет сделать вывод о стабильном преимуществе процессора *Intel Core i5-1235U* во всех тестах. Наиболее значительное превосходство наблюдается в многопоточных сценариях (25,5-35,9%), что обусловлено прогрессом гибридной архитектуры *Alder Lake*. В однопоточных задачах преимущество составляет 9,8-15%, что также свидетельствует о улучшении производительности отдельных ядер. Полученные результаты позволяют предположить, что *i5-1235U* будет демонстрировать более высокую производительность при реализации алгоритма вычисления числа Пи методом Монте-Карло, особенно в многопоточном режиме.

2 ПЛАТФОРМА ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

2.1 Структура и архитектура платформы

Для выполнения данного курсового проекта в качестве программной платформы была выбрана операционная система *Linux*. Это решение обусловлено её высокой стабильностью, эффективным управлением ресурсами и широкими возможностями для разработки программного обеспечения, что является критически важным для проведения точных измерений производительности аппаратного обеспечения.

Операционная система *Linux* имеет монолитную модульную структуру. Это означает, что, несмотря на то что все части ядра функционируют в едином адресном пространстве (что характерно для монолитных ядер), его архитектура спроектирована как набор взаимозаменяемых модулей. Функциональность ОС распределена по модулям, которые могут динамически загружаться и выгружаться по мере необходимости, что обеспечивает гибкость и эффективное использование ресурсов [7].

Основные компоненты архитектуры операционной системы *Linux* представлены на рисунке 2.1.

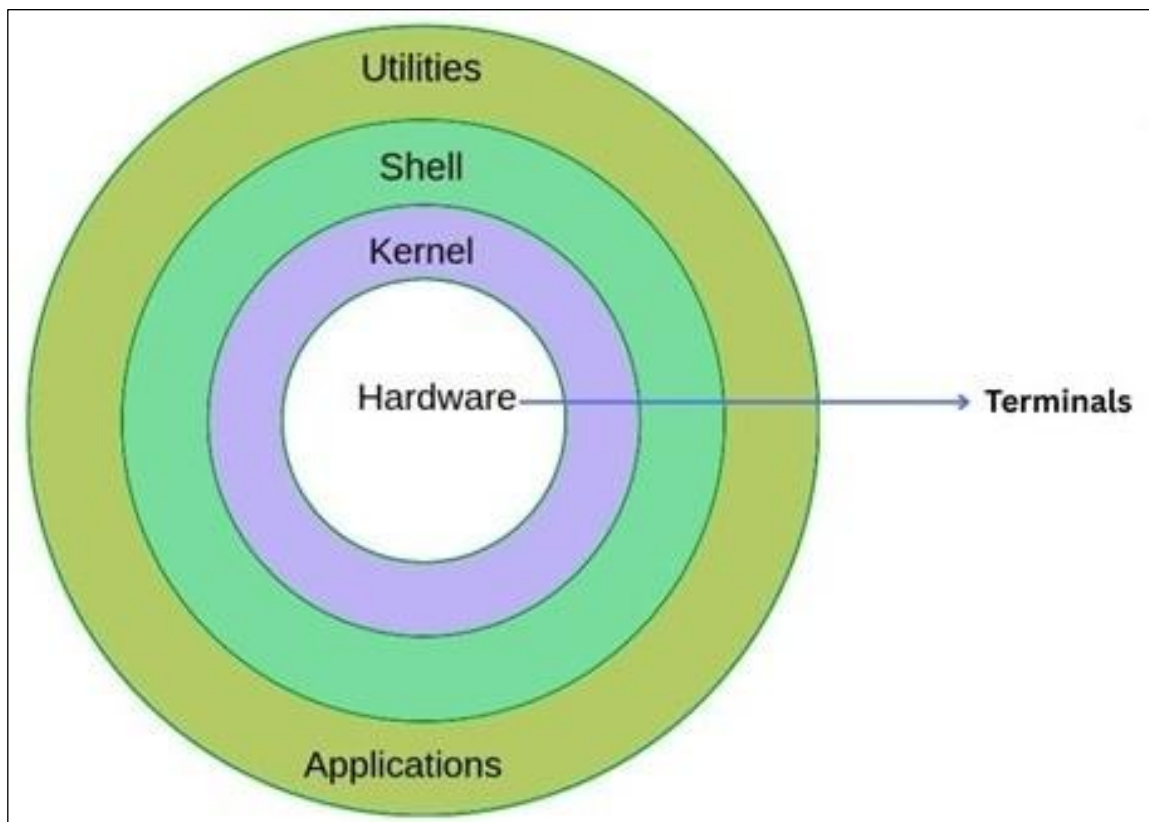


Рисунок 2.1 – Основные компоненты архитектуры операционной системы *Linux*

Основные компоненты и архитектурные особенности платформы *Linux* включают:

1 Ядро *Linux* – это основной компонент, управляющий аппаратным обеспечением и взаимодействием с другими компонентами ОС. Задачи включают управление памятью, процессами и устройствами. Ядро *Linux* представляет собой микроядро, которое обеспечивает базовые функции, включая планирование задач, управление памятью и взаимодействие с устройствами [8].

2 Системные библиотеки – это наборы функций, которые предоставляют приложениям стандартизированный интерфейс для доступа к услугам ядра. Наиболее важной из них является *GNU C Library (glibc)*, без которой невозможна работа большинства приложений в *Linux*. Библиотеки обрабатывают системные вызовы, обеспечивая работу с файлами, сетью, управление памятью и создание потоков (например, через библиотеку *libpthread*).

3 Системные утилиты и инструменты – это программы, выполняющие специализированные задачи управления системой. К ним относятся оболочки командной строки (такие как *bash*), инструменты для обработки текста (*grep*, *sed*, *awk*), компиляторы (*gcc*, *g++*) и система сборки *make*. Набор утилит *BusyBox*, часто используемый в минималистичных средах, предоставляет компактные реализации многих стандартных команд [9].

4 Файловая система. *Linux* поддерживает множество файловых систем, организующих хранение данных на диске. Наиболее распространенными являются: *ext4* – журналируемая файловая система, стандарт де-факто для большинства современных дистрибутивов благодаря своей надежности и стабильности; *XFS* – высокопроизводительная файловая система, хорошо подходящая для работы с большими файлами; *Btrfs* – современная файловая система с поддержкой снапшотов, инкрементального резервного копирования и других продвинутых функций; *Swap* – раздел подкачки, используемый для расширения виртуальной памяти [10].

5 Драйверы устройств. Будучи частью ядра, драйверы обеспечивают абстракцию аппаратного обеспечения, позволяя операционной системе взаимодействовать с физическими устройствами – процессором, памятью, сетевыми адаптерами, накопителями и другими компонентами.

6 Модульность. *Linux* поддерживает модульность, что позволяет добавлять или удалять функциональность ядра динамически во время работы системы. Это позволяет оптимизировать использование ресурсов и обновлять ядро без перезагрузки системы.

На рисунке 2.2 представлена упрощенная структура операционной системы *Linux*.

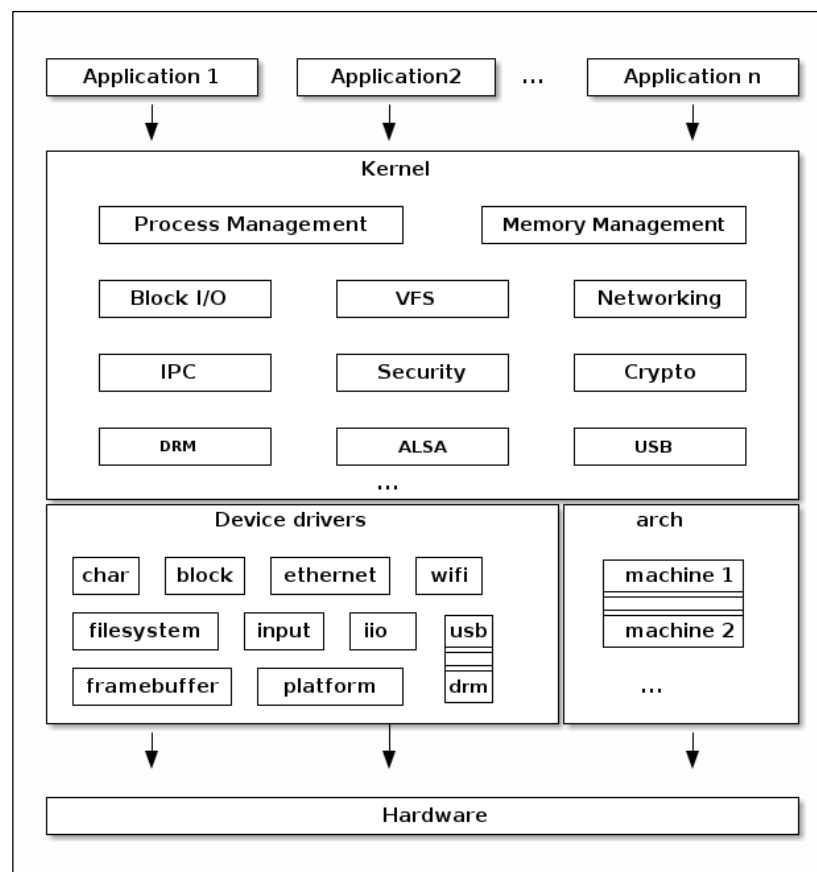


Рисунок 2.2 – Упрощенная структура ОС *Linux*

На рисунке 2.3 продемонстрирована структура ядра операционной системы *Linux*.

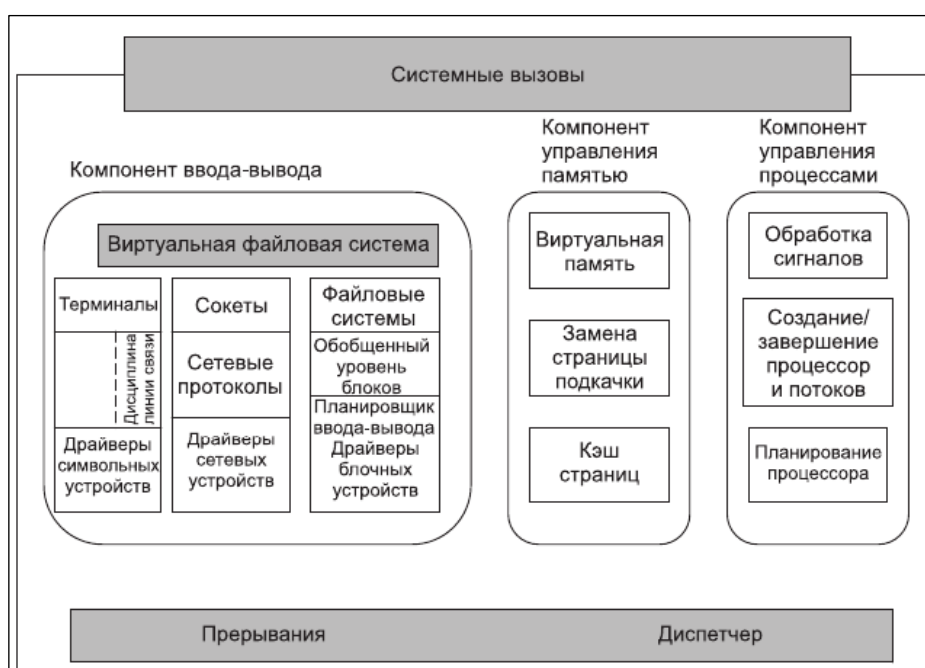


Рисунок 2.3 – Структура ядра операционной системы *Linux*

2.2 История, версии и достоинства

История *Linux* неразрывно связана с развитием движения за свободное программное обеспечение и является ярким примером успешного *Collaborative development* (совместной разработки).

В 1983 году Ричард Столлман инициирует проект *GNU*, целью которого является создание полностью свободной *Unix*-подобной операционной системы. К началу 1990-х годов было создано большинство системных утилит, но не хватало ключевого компонента – ядра [11].

В 1991 году студент Хельсинкского университета Линус Торвальдс начинает разработку собственного ядра операционной системы [12]. Вдохновленный учебной системой *Minix*, он 17 сентября 1991 года публикует в сети первую версию своего ядра, которое позже получает название *Linux*.

Сочетание ядра *Linux* и утилит проекта *GNU* привело к появлению полностью функциональной свободной операционной системы, известной как *GNU/Linux*, а часто просто называемую *Linux*.

Если изначально *Linux* ориентировался на специалистов, то впоследствии дистрибутивы стали более доступными для обычных пользователей. Дистрибутивы различаются не столько по графическому интерфейсу, сколько по области применения – для серверов или настольных компьютеров, а также по способу управления пакетами и другим функциям.

С течением времени *Linux* была адаптирована для различных архитектур и теперь используется в самых разнообразных электронных устройствах.

Версии ядра *Linux* [13]:

Версия 1.x (1991 – 1995): версия ядра *Linux*, созданная Линусом Торвальдсом. Включала базовые функции, такие как управление процессами, файловой системой и базовое взаимодействие с оборудованием.

Версия 2.x (1996 – 2003): поддержка многопроцессорных систем и улучшенное управление памятью, улучшение сетевого стека и масштабируемости, поддержка новых файловых систем, таких как *ext3*.

Версия 3.x (2011 – 2013): объединение ряда нововведений, включая поддержку улучшенных драйверов, более быстрые файловые системы и новые механизмы планирования задач.

Версия 4.x (2015 – 2019): повышенная энергоэффективность, поддержка файловой системы *Btrfs*, развитие технологий сетей и драйверов.

Версия 5.x (2019 – 2022): расширенная поддержка архитектур *ARM*, *RISC-V*, новые функции безопасности.

Версия 6.x (2022 – настоящее время): дальнейшая оптимизация для современных серверных и встраиваемых систем, улучшения в производительности и виртуализации.

2.3 Обоснование выбора платформы

В рамках данного курсового проекта было принято решение

использовать *Linux Ubuntu* и этот выбор обоснован следующими факторами:

1 Стабильность и предсказуемость среды: *Ubuntu LTS (Long-Term Support)* предлагает стабильные и проверенные версии программного обеспечения с длительной поддержкой. Это позволяет создать неизменяемую и воспроизводимую среду для проведения измерений, что критически важно для получения достоверных и сравниваемых результатов тестирования производительности процессоров. Минимизация влияния сторонних факторов и сбоев обеспечивает чистоту эксперимента.

2 Широкая поддержка оборудования и оптимизация: как один из самых популярных дистрибутивов, *Ubuntu* обеспечивает отличную совместимость с современным аппаратным обеспечением, включая процессоры *Intel* 12-го и 11-го поколений. Дистрибутив регулярно получает обновления микрокода и оптимизации ядра, направленные на повышение производительности и энергоэффективности на актуальных платформах, что позволяет в полной мере раскрыть потенциал тестируемых процессоров.

3 Доступ к инструментам разработки: Репозитории *Ubuntu* содержат актуальные версии компиляторов (*GCC*, *Clang*), систем сборки (*make*, *CMake*) и отладчиков (*gdb*). Это предоставляет все необходимые инструменты для эффективной разработки, оптимизации и отладки приложения на C++, а также для точного анализа его работы на уровне процессора.

4 Основы на ядре *Linux* и стандартизация: *Ubuntu*, будучи основанной на ядре *Linux*, предоставляет все преимущества этой операционной системы: эффективное планирование задач для многоядерных процессоров, развитую подсистему управления памятью и прозрачность работы. Широкая распространенность *Ubuntu* делает ее де-факто стандартом для многих разработчиков, что гарантирует совместимость и надежность используемого программного стека.

Таким образом, выбор *Linux Ubuntu LTS* в качестве платформы для тестирования был сделан на основе ее стабильности, отличной поддержки оборудования, доступности необходимых инструментов разработки и предсказуемости работы. Эти преимущества делают *Ubuntu* оптимальной средой для проведения точных и объективных сравнительных измерений производительности процессоров.

2.4 Анализ операционной системы для написания программы

Ubuntu – это один из самых популярных и широко используемых дистрибутивов *Linux*, разрабатываемый компанией *Canonical*. Её философия заключается в предоставлении свободной, удобной и стабильной операционной системы как для обычных пользователей, так и для корпоративной среды и разработчиков.

Ключевые преимущества *Ubuntu* для выполнения вычислительных экспериментов:

1 Широкая популярность и поддержка: *Ubuntu* является стандартом во

многих сферах, от облачных вычислений до разработки программного обеспечения. Это означает, что практически любое программное обеспечение, драйверы или библиотеки в первую очередь разрабатываются и тестируются именно для *Ubuntu*. Такая распространенность гарантирует максимальную совместимость, упрощает поиск решений проблем и обеспечивает доступ к обширной базе знаний сообщества.

2 Готовность к работе и обширные репозитории: после установки *Ubuntu* предоставляет полностью настроенную систему с широким набором предустановленных приложений и инструментов. Его репозитории содержат тысячи пакетов, что позволяет легко установить необходимое программное обеспечение для разработки (*IDE*, компиляторы, библиотеки) всего несколькими командами, не тратя время на сложную настройку окружения.

3 Стабильность и производительность: используя долгосрочную версию (*LTS*), пользователь получает крайне стабильную и отлаженную систему. Ядро *Linux*, лежащее в основе *Ubuntu*, оптимизировано для эффективной работы с многоядерными процессорами и многопоточными приложениями. Менеджер пакетов *apt* и система инициализации *systemd* обеспечивают надежное управление программным обеспечением и системными сервисами, что важно для поддержания целостности экспериментальной среды.

4 Гибкость и производительность: несмотря на богатую базовую функциональность, *Ubuntu* остается достаточно легковесной системой, особенно при использовании серверной редакции или минималистичных окружений рабочего стола. Это позволяет максимально направить ресурсы тестируемых процессоров на выполнение целевой задачи – вычисление числа Π методом Монте-Карло, минимизировав фоновую нагрузку от операционной системы.

Благодаря сочетанию стабильности, отличной поддержки инструментов разработки, эффективного управления ресурсами и широкого сообщества, *Ubuntu* представляет собой идеально подогнанную платформу для проведения ресурсоемких вычислений и сравнительного анализа производительности аппаратного обеспечения.

3 ТЕОРЕТИЧЕСКОЕ ОБОСНОВАНИЕ РАЗРАБОТКИ ПРОГРАММНОГО ПРОДУКТА

3.1 Обоснование необходимости разработки

В современных вычислительных системах производительность процессоров играет ключевую роль в обеспечении эффективного решения различных научных и инженерных задач. Многоядерные процессоры, такие как *Intel Core i5-1235U* и *Intel Core i5-1135G7*, представляют собой стандарт для современных устройств, и выбор оптимального процессора для конкретных вычислительных задач при ограниченном бюджете остается актуальной проблемой.

Анализ производительности вычислений на многоядерных процессорах при реализации вероятностных алгоритмов, в частности метода Монте-Карло для вычисления числа Π , представляет собой важную исследовательскую задачу в условиях роста сложности вычислительных проблем. Это обосновано следующими ключевыми факторами:

1 Оптимизация параллельных вычислений. Метод Монте-Карло идеально подходит для распараллеливания, поскольку отдельные испытания являются независимыми. Эффективное использование многоядерной архитектуры современных процессоров позволяет значительно сократить время вычислений при увеличении количества итераций, что особенно важно для достижения высокой точности результатов.

2 Практическая значимость вероятностных методов. Методы Монте-Карло находят широкое применение в таких областях, как финансовое моделирование, физическое моделирование, криптография и машинное обучение. Анализ производительности процессоров при выполнении подобных алгоритмов имеет непосредственное практическое значение для оптимизации вычислительных процессов в этих областях.

3 Сравнительный анализ архитектурных решений. Процессоры *Intel Core i5-1235U* (гибридная архитектура *Alder Lake*) и *i5-1135G7* (классическая однородная архитектура *Tiger Lake*) представляют разные подходы к организации вычислений. Исследование их поведения при выполнении однотипных параллельных вычислений позволяет оценить эффективность архитектурных инноваций в реальных сценариях использования.

4 Обоснование выбора аппаратного обеспечения. Для научных работников, инженеров и разработчиков программного обеспечения важно понимать, как различные процессорные архитектуры справляются с задачами, требующими массовых параллельных вычислений. Результаты сравнительного анализа предоставляют ценную информацию для принятия обоснованных решений при выборе вычислительных платформ.

Таким образом, разработка программы для сравнения производительности процессоров *Intel Core i5-1235U* и *i5-1135G7* при

вычислении числа Пи методом Монте-Карло не только важна с точки зрения оценки технологического прогресса, но и имеет практическую ценность для оптимизации вычислительных процессов в различных предметных областях.

3.2 Технологии программирования, используемые для решения поставленных задач

Для реализации программы, направленной на исследование производительности процессоров при вычислении числа Пи методом Монте-Карло, был выбран язык программирования C++. Данный выбор обоснован следующими факторами, связанными с особенностями реализации вероятностных алгоритмов и требованиями к точности измерений.

C++ обеспечивает высокую производительность и детальный контроль над управлением памятью, что критически важно для итеративных алгоритмов, требующих выполнения большого количества однотипных операций. При реализации метода Монте-Карло это позволяет минимизировать накладные расходы и точно измерять производительность непосредственно вычислительных этапов алгоритма, а не вспомогательных операций.

Для распараллеливания вычислений использовалась технология *OpenMP*, которая предоставляет удобные директивы для параллелизации циклов на уровне исходного кода. В контексте метода Монте-Карло это особенно эффективно, поскольку каждая итерация алгоритма (генерация случайной точки и проверка её попадания в круг) является независимой. Использование директив *#pragma omp parallel* и *#pragma omp for* [15] позволило равномерно распределить вычислительную нагрузку между всеми доступными потоками процессоров, что существенно ускорило выполнение алгоритма при увеличении количества испытаний.

Библиотека *<chrono>*, входящая в стандарт C++, была использована для высокоточного измерения времени выполнения алгоритма. С помощью *std::chrono::high_resolution_clock* и *std::chrono::duration_cast* обеспечивается точность измерений на уровне наносекунд, что необходимо для корректного сравнения производительности процессоров при выполнении коротких вычислительных циклов, характерных для метода Монте-Карло.

Для генерации псевдослучайных чисел использовался современный генератор *std::mt19937* (Вихрь Мерсенна) из библиотеки *<random>*, который обеспечивает хорошее статистическое качество случайных последовательностей и высокую скорость работы, что важно для получения достоверных результатов при большом количестве испытаний.

Для сохранения результатов расчетов применялась библиотека *<fstream>*, которая обеспечила запись данных в формате CSV. Это позволило фиксировать не только общее время выполнения, но и промежуточные результаты при различных количествах итераций, что необходимо для последующего анализа масштабируемости алгоритма на разных процессорах.

Особое внимание было уделено управлению аффинити потоков с использованием системных вызовов *Linux*. Для этого применялась функция *pthread_setaffinity_np*, которая позволяет явно привязывать потоки к конкретным ядрам процессора. Это особенно важно для исследования гибридной архитектуры процессора *Intel Core i5-1235U*, где можно отдельно проанализировать производительность на производительных и энергоэффективных ядрах. Использование структур *cpu_set_t* и макроса *CPU_SET* обеспечило точный контроль над распределением вычислительной нагрузки.

Для компиляции программы использовался компилятор *GCC* с ключами оптимизации *-O2* и *-march=native*, что гарантирует генерацию эффективного машинного кода, учитывающего особенности микроархитектуры каждого тестируемого процессора.

Таким образом, комбинация *C++*, *OpenMP*, современных библиотек *C++* и средств управления потоками *Linux* предоставила необходимый инструментарий для проведения точных и воспроизводимых измерений производительности процессоров при выполнении алгоритма Монте-Карло, учитывая все особенности их архитектурных решений.

3.3 Связь архитектуры вычислительной системы с разрабатываемым программным обеспечением

Разрабатываемая программа тесно интегрирована с архитектурными особенностями тестируемых процессоров через использование библиотеки *hwloc* для анализа топологии и управления аффинити потоков. Реализация алгоритма вычисления Π методом Монте-Карло предусматривает несколько режимов выполнения: последовательную обработку, многопоточные вычисления с использованием *OpenMP*, работу на отдельных физических ядрах и *GPU*-ускорение через *OpenCL*. Такой подход позволяет исследовать влияние различных архитектурных конфигураций на производительность при выполнении вероятностных вычислений.

Программа обеспечивает детальный анализ процессорной топологии, выявляя количество физических ядер и их распределение, что позволяет проводить целевое тестирование производительности при работе на конкретных ядрах процессора. Комбинация технологий *OpenMP* для параллельных вычислений и средств управления аффинити потоков через *hwloc* обеспечивает точное распределение вычислительной нагрузки. Программа позволяет исследовать эффективность работы как на однородных, так и на неоднородных архитектурах через механизм привязки к конкретным физическим ядрам.

Ключевые аспекты взаимодействия программного обеспечения с архитектурой процессора:

1 Физическое ядро – аппаратная единица выполнения команд, обладающая собственными вычислительными ресурсами.

2 Логическое ядро – виртуальный процессор, создаваемый технологией Hyper-Threading для одновременного выполнения нескольких потоков на одном физическом ядре. Оба тестируемых процессора поддерживают *Hyper-Threading*, предоставляя 12 и 8 логических ядер соответственно.

3 Процесс – изолированная программа, работающая в своем адресном пространстве, которая не имеет доступа к памяти других процессов.

4 Поток – это базовая единица, которой операционная система выделяет процессорное время. Поток может выполнять любую часть кода процесса, включая те части, которые в настоящее время выполняются другим потоком [16].

Для точного контроля распределения вычислений используется механизм *CPU Affinity* – программная привязка потоков к конкретным ядрам процессора. Это исключает "переключение контекста" между ядрами и обеспечивает стабильность измерений. В программе применяется функция *pthread_setaffinity_np* из библиотеки *pthread* в *Linux* для статического закрепления потоков на заданных ядрах [17].

Технология *OpenMP* обеспечивает эффективное распараллеливание основного цикла алгоритма, где каждая итерация (генерация и проверка случайной точки) выполняется независимо. Директивы *#pragma omp parallel for* позволяют динамически распределять итерации между доступными потоками, максимизируя загрузку процессорных ресурсов.

Проведенное исследование и разработка программного продукта позволяют глубоко проанализировать особенности работы современных многоядерных процессоров с различной архитектурой и их влияние на производительность при выполнении вероятностных алгоритмов. Используемые технологии программирования обеспечивают необходимую гибкость, точность и эффективность проводимых вычислений.

4 ПРОЕКТИРОВАНИЕ ФУНКЦИОНАЛЬНЫХ ВОЗМОЖНОСТЕЙ ПРОГРАММЫ

4.1 Общая цель программы

Цель программы заключается в проведении сравнительного анализа производительности двух процессоров – *Intel Core i5-1235U* и *Intel Core i5-1135G7* при вычислении числа Π с использованием вероятностного метода Монте-Карло. Программа позволяет оценить эффективность различных вычислительных, обеспечивая сравнительный анализ производительности между последовательными вычислениями, многопоточными вычислениями на *CPU* с использованием *OpenMP*, *GPU*-ускорением через *OpenCL* и работой на отдельных физических ядрах процессора. С помощью механизмов управления аффинностью потоков и детального анализа топологии процессора программа выявляет особенности работы вычислительной системы при различных сценариях нагрузки: от полной загрузки всех доступных ядер до целенаправленного использования конкретных физических ядер, что позволяет исследовать влияние распределения вычислительной нагрузки на общую производительность системы.

Программа обеспечивает:

- вычисление числа Π методом Монте-Карло с различным количеством испытаний;
- тестирование производительности в различных режимах работы процессора;
- измерение времени выполнения вычислений с высокой точностью;
- сравнение эффективности распараллеливания вычислений;
- сохранение результатов тестирования для последующего анализа.

Результаты работы программы предоставляют ценную информацию для выбора оптимального процессорного решения для задач, требующих интенсивных параллельных вычислений, а также демонстрируют практическую эффективность архитектурных инноваций в современных процессорах.

4.2 Теоретические сведения о методе Монте-Карло

Методы вычисления математических констант можно разделить на две группы: аналитические (детерминистические) методы и вероятностные (статистические) методы.

К аналитическим методам относят методы, которые основаны на строгих математических формулах и позволяют получить точное значение константы с заданной точностью. К вероятностным методам относят методы, которые используют случайные процессы и позволяют получить приближенное значение константы со статистической погрешностью.

Метод Монте-Карло является одним из наиболее распространенных вероятностных методов вычисления числа π и относится к классу статистических методов [18].

Основная идея метода Монте-Карло для вычисления числа π основана на геометрической интерпретации. Рассмотрим круг, вписанный в единичный квадрат. Площадь круга равна $\pi/4$, а площадь квадрата равна 1.

Если мы будем случайным образом генерировать точки внутри квадрата, то отношение количества точек, попавших внутрь круга, к общему количеству точек будет приблизительно равно отношению площадей:

$$\frac{N_{inside}}{N_{total}} \approx \frac{\pi}{4}$$

Отсюда получаем формулу для приближенного вычисления π :

$$\pi \approx 4 \times \frac{N_{inside}}{N_{total}}, \quad (4.1)$$

где:

– N_{inside} – количество точек, попавших внутрь круга;

– N_{total} – общее количество сгенерированных точек.

Уравнение круга диаметра 1 с центром в начале координат имеет вид:

$$x^2 + y^2 \leq 1, \quad (4.2)$$

где x и y – координаты точки.

Для каждой случайной точки (x, y) , равномерно распределенной в квадрате $[-1, 1] \times [-1, 1]$, проверяется условие (4.2). Если условие выполняется, точка считается попавшей внутрь круга.

Вероятность того, что случайная точка попадет внутрь круга, равна отношению площадей:

$$P = \frac{S_{circle}}{S_{square}} = \frac{\pi \times \frac{1}{2^2}}{1} = \frac{\pi}{4}$$

По закону больших чисел, при увеличении количества испытаний N_{total} , частота попаданий $\frac{N_{inside}}{N_{total}}$ сходится к вероятности P :

$$\lim_{N_{total} \rightarrow \infty} \frac{N_{inside}}{N_{total}} = P = \frac{\pi}{4}$$

Алгоритм вычисления числа π методом Монте-Карло состоит из следующих шагов:

- 1 Инициализация: задается общее количество точек N_{total} .
- 2 Генерация случайных точек для i от 1 до N_{total} : генерируются случайные координаты x_i и y_i в диапазоне $[0, 1]$.
- 3 Для каждой точки проверяется условие (4.2).
- 4 Увеличивается счетчик N_{inside} , если условие выполняется.
- 5 Вычисляется приближенное значение π по формуле (4.1).

Погрешность метода Монте-Карло убывает пропорционально $\frac{1}{\sqrt{N}}$, где N – количество испытаний. Это означает, что для уменьшения погрешности в 10 раз необходимо увеличить количество испытаний в 100 раз.

Стандартная ошибка вычисления π оценивается по формуле:

$$\sigma_{\pi} = 4 \times \sqrt{\frac{P(1 - P)}{N}} \approx \sqrt{\frac{\pi (4 - \pi)}{N}},$$

где P – вероятность попадания точки в круг.

Для больших N распределение оценки π является приближенно нормальным с математическим ожиданием π и стандартным отклонением σ_{π} .

Метод Монте-Карло особенно эффективен для решения задач высокой размерности, где традиционные методы становятся вычислительно дорогими. В случае вычисления числа π метод представляет не только практический, но и образовательный интерес, наглядно демонстрируя связь между геометрией и теорией вероятностей.

4.3 Описание функциональных возможностей

Программа представляет собой комплексную систему бенчмаркинга для сравнительного анализа производительности вычисления числа π методом Монте-Карло на различных вычислительных архитектурах. Она реализует многоуровневую архитектуру вычислений с поддержкой последовательной обработки, *OpenMP* мультипоточности на *CPU*, *GPU*-ускорения через *OpenCL* и специализированного режима работы на одном физическом ядре процессора.

Система автоматически обнаруживает и анализирует аппаратные ресурсы компьютера, используя библиотеку *hwloc* для определения топологии процессора, включая количество физических ядер, распределение логических процессоров и иерархическую структуру кэш-памяти, а также интеллектуально выбирает доступные *GPU*.

Программа обеспечивает измерение производительности с микросекундной точностью, используя библиотеку *chrono*, и гарантирует статистическую достоверность результатов за счет многократного

выполнения тестов с последующим усреднением показателей и расчетом относительной погрешности вычислений.

Система предоставляет гибкие сценарии тестирования, включая автоматический режим с предустановленными наборами тестов до 5 миллионов точек, и расширенный ручной режим с возможностью настройки количества точек, числа итераций и выбора тестируемых методов вычисления.

Управление процессорной аффинностью обеспечивает точную привязку потоков к конкретным физическим ядрам через *hwloc*, что исключает влияние планировщика операционной системы на результаты измерений и оптимизирует использование кэш-памяти.

Комплексная система отчетности формирует детализированные таблицы с информацией о времени выполнения, полученном значении Π , относительной погрешности и количестве использованных потоков, а также обеспечивает экспорт результатов в *CSV* формат для последующего анализа во внешних инструментах.

Интерактивный пользовательский интерфейс с консольным меню предоставляет интуитивную навигацию.

В целом, программа обеспечивает комплексный подход к анализу производительности процессоров при вычислении числа Π методом Монте-Карло. Реализованные функциональные возможности позволяют эффективно проводить тестирование, учитывать влияние параллелизации, а также исследовать зависимость точности и скорости вычислений от объема обрабатываемых данных.

5 СРАВНЕНИЕ ПРОИЗВОДИТЕЛЬНОСТИ ПРОЦЕССОРОВ

5.1 Общая структура программы

Программа представляет собой систему для сравнительного анализа производительности вычисления числа Π методом Монте-Карло на различных вычислительных архитектурах, включая последовательное выполнение на *CPU*, многопоточное выполнение через *OpenMP*, выполнение на одном физическом ядре процессора и *GPU*-ускоренные вычисления через *OpenCL*. Листинг программного кода представлен в приложении Б.

Архитектура программы построена по модульному принципу и включает несколько ключевых компонентов. Основу программы составляет структура данных *PiResults*, которая инкапсулирует результаты тестирования, включая время вычислений в микросекундах, полученное значение Π , относительную погрешность в процентах и количество использованных потоков.

Модуль *GPU*-вычислений реализован в классе *GPUPiCalculator*, который обеспечивает автоматическое обнаружение доступных *GPU* устройств, инициализацию *OpenCL* контекста и выполнение массово-параллельных вычислений через специализированный кернел *monte_carlo_pi*. Класс автоматически инициализируется при создании глобального объекта *g_gpu_calculator* до вызова функции *main*, что обеспечивает готовность *GPU* к использованию на момент начала работы программы. Конструктор класса выполняет поиск *OpenCL* платформ и устройств типа *GPU* с предпочтением *Intel iGPU*, создает контекст и очередь команд, компилирует *OpenCL kernel* из встроенного исходного кода и обрабатывает ошибки через механизм исключений с установкой флага доступности *GPU*.

Модуль управления аппаратной топологией использует библиотеку *hwloc* для анализа процессорной архитектуры и предоставляет функции для точной привязки потоков к физическим ядрам. Глобальная переменная *topology* хранит структуру топологии системы. Функция *initTopology* инициализирует систему обнаружения аппаратных ресурсов через вызовы *hwloc_topology_init* и *hwloc_topology_load*, *setAffinityToPhysicalCore* обеспечивает привязку текущего потока к конкретному физическому ядру через *hwloc_set_cpubind*, а *printTopologyInfo* выводит детальную информацию о конфигурации процессора, включая количество физических ядер и соответствие логических *CPU* каждому ядру. Функция *cleanupTopology*

освобождает ресурсы топологии перед завершением программы.

Генерация тестовых данных осуществляется с использованием генератора псевдослучайных чисел `std::mt19937` с равномерным распределением `std::uniform_real_distribution<float>` в диапазоне `[0, 1]` для CPU-реализаций. Для последовательного метода используется один генератор на весь процесс, для *OpenMP* каждый поток создает собственный генератор с уникальным *seed*, что обеспечивает независимость генерации в параллельных потоках. GPU-реализация использует линейный конгруэнтный генератор (LCG) на стороне устройства с уникальным *seed* для каждого рабочего элемента.

Система тестирования построена на шаблонной функции *TestPiTimed*, которая обеспечивает многократное выполнение тестов с последующим усреднением результатов для повышения статистической достоверности. Программа предоставляет два режима тестирования: автоматический и ручной.

Автоматический режим выполняет прогрессивное тестирование на 27 различных размерах данных от примерно 185 тысяч до 5 миллионов точек с равномерным шагом. Каждый тест выполняется в 10 итераций, и все четыре метода тестируются автоматически. Ручной режим позволяет пользователю выбрать источник параметров теста (предустановленные значения из набора 100 тысяч до 500 миллионов точек или произвольные значения), количество итераций от 1 до 50 и индивидуально выбрать методы тестирования через интерактивные вопросы да/нет.

Параллельная реализация *OpenMP* использует директиву `#pragma omp parallel reduction(+ : points_inside)` для безопасного параллельного суммирования точек внутри круга, и директиву `#pragma omp for` для распределения итераций цикла между потоками. Программа автоматически определяет максимальное количество доступных потоков через `omp_get_max_threads()` и устанавливает его как значение по умолчанию через `omp_set_num_threads`.

Для обеспечения точности временных измерений используется библиотека `std::chrono` с высокоточными часами `std::chrono::high_resolution_clock`. Время измеряется в микросекундах через преобразование `std::chrono::duration_cast<std::chrono::microseconds>`, что обеспечивает достаточную точность для различения производительности различных методов.

Модуль вывода результатов предоставляет два формата представления данных: форматированные таблицы в консоли и структурированный CSV-

файл для последующего анализа. Таблица содержит информацию о методе, количестве потоков, времени выполнения, значении Π и относительной погрешности. Функция *saveResultsToFile* сохраняет полный набор результатов в CSV-файл с разделителями-запятыми, включая данные для всех протестированных методов при каждом размере данных.

Управление пользовательским интерфейсом осуществляется через консольное меню, отображаемое функцией *showMainMenu*, которое предоставляет четыре опции: автоматический тест, ручная настройка теста, информация о системе и выход. Главный цикл программы построен на конструкции *do-while* с обработкой выбора пользователя через *switch-case*. Ввод данных пользователем валидируется функциями *getIntegerInput*, *getLongLongInput* и *getYesNoInput*, которые обеспечивают обработку ошибок ввода и повторный запрос при некорректных данных. Графический интерфейс пользователя представлен в приложении Е.

5.2 Описание функциональной схемы программы

Функциональная схема программы представлена в приложении В. В ходе разработки функциональной схемы выделены следующие ключевые блоки:

1 Инициализация системы. Данный блок выполняет подготовку вычислительной среды к выполнению тестирования. Он осуществляет инициализацию топологии процессора через библиотеку *hwloc* для идентификации физических ядер и логических процессоров, выводит детальную информацию о процессорной архитектуре, определяет максимальное количество доступных потоков *OpenMP* и устанавливает его как значение по умолчанию. Блок проверяет доступность *GPU* функциональности через глобальный объект *g_gpu_calculator*, который автоматически инициализируется до вызова функции *main*, и информирует пользователя о состоянии графических ускорителей.

2 Выбор режима работы. Блок реализует интерактивное управление программой через консольное меню, предоставляющее пользователю четыре варианта работы: автоматический тест с рекомендуемыми параметрами, ручная настройка теста, вывод информации о системе и выход из программы. Главный цикл обеспечивает непрерывную работу программы до явного выбора пользователя о завершении. Обработка выбора осуществляется через конструкцию *switch-case*, которая направляет выполнение к соответствующему функциональному модулю. Блок обеспечивает интуитивный пользовательский интерфейс с валидацией ввода, гарантируя корректность выбранных опций.

3 Настройки параметров теста. Данный блок выполняет конфигурацию

параметров тестирования в зависимости от выбранного режима работы. В автоматическом режиме блок генерирует 27 различных размеров данных от примерно 185 тысяч до 5 миллионов точек с равномерным распределением и устанавливает фиксированное количество итераций равным 10, при этом все четыре метода тестирования активируются автоматически. В ручном режиме блок предоставляет пользователю возможность выбора источника параметров: использование предустановленных значений от 100 тысяч до 500 миллионов точек или ввод произвольных значений в диапазоне от 1000 до 1 миллиарда точек. Блок позволяет настроить количество итераций от 1 до 50 для усреднения результатов и индивидуально выбрать методы тестирования через интерактивные вопросы: последовательный метод, *OpenMP*, метод одного ядра и *GPU*-вычисления.

4 Выполнение вычислений. Центральный вычислительный блок программы реализует четыре различных подхода к вычислению числа Π методом Монте-Карло с многократным повторением для статистической достоверности. Последовательная версия обеспечивает базовые однопоточные вычисления с использованием генератора *std::mt19937* и равномерного распределения в диапазоне $[0,1]$. *OpenMP* реализация задействует все доступные ядра процессора через директивы *parallel reduction* и *for*, где каждый поток использует независимый генератор случайных чисел с уникальным *seed*. *GPU*-ускоренная версия использует массовый параллелизм графических процессоров через *OpenCL kernel* с линейным конгруэнтным генератором на стороне устройства. Режим одного ядра позволяет исследовать производительность отдельных физических ядер через механизм привязки потоков с использованием *hwloc*. Все вычисления выполняются с заданным количеством итераций, обеспечивая многократное выполнение с последующим усреднением результатов. Временные измерения выполняются с высокой точностью через *std::chrono::high_resolution_clock* с преобразованием в микросекунды.

5 Обработка результатов. Блок осуществляет статистическую обработку и агрегацию полученных данных тестирования. Функции тестирования выполняют накопление сумм времени выполнения, значений Π и относительной погрешности по всем итерациям теста. По завершении цикла итераций блок вычисляет средние значения путем деления накопленных сумм на количество итераций, что обеспечивает повышение статистической достоверности результатов и минимизацию влияния случайных факторов. Блок рассчитывает относительную погрешность для каждого метода.

6 Вывод результатов. Данный блок преобразует обработанные результаты измерений в удобочитаемый формат для визуализации и анализа. Блок формирует структурированные таблицы с выравниванием колонок, включающие заголовок с общей информацией о тесте, колонки с наименованием метода, количеством использованных потоков, временем выполнения в микросекундах, вычисленным значением Π с высокой точностью и относительной погрешностью в процентах. Блок отображает

результаты в консоли после каждого размера теста, обеспечивая немедленную обратную связь с пользователем.

7 Завершение работы. Финальный блок системы обеспечивает корректное завершение работы программы и освобождение выделенных ресурсов. Блок проверяет наличие запроса на сохранение результатов в файл и экспортирует полный набор данных тестирования в структурированный CSV-файл с разделителями-запятыми, включая параметры экспериментов, характеристики методов, временные показатели и расчетные значения с указанием относительной погрешности. Блок вызывает функцию освобождения ресурсов топологии *hwloc*, что гарантирует корректное завершение работы на различных аппаратных платформах.

5.3 Описание блок-схемы алгоритма программы

Блок-схема алгоритма программы представлена в приложении Г. Алгоритм реализован в виде последовательной схемы с циклическими структурами и условными переходами.

Алгоритм начинается с блока инициализации системы, который выполняет подготовку вычислительной среды: инициализацию топологии процессора, определение параметров *OpenMP* и проверку доступности *GPU*. После завершения инициализации управление передается к главному циклу программы.

Главный цикл реализован через условную структуру с условием продолжения работы до выбора пользователем опции выхода. Внутри цикла выполняется отображение меню, ввод выбора пользователя с валидацией и проверка выбранного варианта через структуру принятия решений. В зависимости от выбора осуществляется переход к одному из четырех функциональных модулей: автоматическому тесту, ручной настройке, выводу информации о системе или завершению работы.

Блок настройки параметров теста имеет два варианта реализации: автоматический режим с предопределенными параметрами и ручной режим с интерактивным вводом. В обоих режимах формируется набор размеров данных для тестирования и количество итераций для усреднения результатов.

Алгоритм выполнения вычислений организован в виде двух вложенных циклов: внешнего цикла по количеству тестовых размеров и внутреннего цикла итераций для статистического усреднения. Для каждого выбранного метода вычисления (последовательный, *OpenMP*, одно ядро, *GPU*) выполняется генерация случайных точек, подсчет точек внутри единичного круга, вычисление значения Π и измерение времени выполнения. После завершения цикла итераций производится усреднение результатов по времени, значению Π и относительной погрешности.

Блок обработки результатов агрегирует данные всех методов тестирования, вычисляет средние значения и относительную погрешность. Блок вывода результатов формирует структурированные таблицы с

результатами для отображения в консоли после каждого размера теста.

Алгоритм завершения работы проверяет необходимость сохранения результатов в файл и выполняет экспорт данных в CSV-формат. После обработки всех данных осуществляется освобождение ресурсов топологии и завершение программы.

5.4 Подготовка к тестированию

Для анализа производительности будет проведено тестирование на процессорах в различных режимах загрузки. Основной акцент будет сделан на сравнении скорости работы при выполнении на одном ядре по сравнению с полной загрузкой процессора. Испытания проводятся в два этапа: с включенным *HyperThreading* и с отключенным, чтобы оценить влияние этого фактора на эффективность вычислений.

Размеры тестовых данных, на которых проводились измерения, варьировались от 185185 до 5000000 элементов с равным шагом. Каждое измерение времени выполнения запускается по 10 раз для минимизации влияния случайных системных факторов. В качестве метрики для анализа выбрано среднее время выполнения тестов, которое позволяет объективно оценить, как различные режимы загрузки влияют на итоговую производительность при выполнении поисковых операций.

Сравнение проводилось в следующих режимах:

- последовательный алгоритм без использования *OpenMP*;
- параллельный алгоритм с использованием *OpenMP*;
- последовательный алгоритм с привязкой к одному ядру процессора
- параллельное выполнение на *iGPU* с использованием *OpenCL*.

Исследования начались с проведения тестов на обоих устройствах с включенным *HyperThreading*. После этого функция *HyperThreading* была отключена, и тесты были повторно запущены для получения следующего набора данных.

5.5 Результаты измерений с включенным *Hyper-Threading*

Таблица 5.1 представляет замеры времени выполнения (в микросекундах) алгоритма вычисления числа Π методом Монте-Карло на процессоре *Intel Core i5-1235U* с включенным *Hyper-Threading*. В таблице представлены результаты для различных объемов вычислительных данных.

Колонка "Количество точек" указывает общее количество сгенерированных случайных точек для вероятностного вычисления. В колонке "Последовательный" представлено время выполнения базового алгоритма Монте-Карло без использования параллельных технологий. Колонка "Параллельный (*OpenMP*)" отображает результаты многопоточной реализации с автоматическим распределением нагрузки между доступными

ядрами процессора. Колонка "На одном ядре" демонстрирует время выполнения последовательного алгоритма с привязкой к конкретному физическому ядру процессора с использованием библиотеки *hwloc*. Колонка "На GPU" показывает время выполнения массово-параллельных вычислений на графическом ускорителе через технологию *OpenCL*.

Таблица 5.1 – Замеры времени вычисления Пи методом Монте-Карло на *Intel Core i5-1235U* с включенным *Hyper-Threading*

Количество точек	Последовательный	Параллельный (OpenMP)	На одном ядре	На GPU
185185	1922	424	2275	850
370370	3317	532	3202	765
555555	4784	719	4766	1038
740740	6379	936	6365	1325
925925	7970	1120	8001	1538
1111110	9545	1358	9566	1788
1296295	11138	1558	11115	2132
1481480	12740	1767	12751	2450
1666665	14321	1986	14309	2880
1851850	15943	2175	15947	2786
2037035	17536	2433	17520	3151
2222220	19126	2620	19107	2307
2407405	20715	2818	20744	2509
2592590	22291	3086	22509	2699
2777775	24122	3577	23831	2827
2962960	25495	3524	25766	2973
3148145	27363	3886	27117	3234
3333330	28668	3939	28657	3367
3518515	30202	4202	30259	3612
3703700	31814	4504	31846	3811
3888885	33395	4960	33423	4090
4074070	34978	4799	35051	4375
4259255	36616	5256	36662	4703
4444440	38180	5326	38163	4798
4629625	39810	5515	39841	5037
4814810	41416	6343	41418	5309
5000000	42906	6004	43031	5532

Таблица 5.2 представляет замеры времени на процессоре *Intel Core i5-1135G7* с включенной функцией *Hyper-Threading*.

Таблица 5.2 – Замеры времени вычисления Пи методом Монте-Карло на *Intel Core i5-1135G7* с включенным *Hyper-Threading*

Количество точек	Последовательный	Параллельный (<i>OpenMP</i>)	На одном ядре	На <i>GPU</i>
185185	1922	482	2046	466
370370	3475	906	3617	636
555555	4713	1285	5183	947
740740	6280	1687	6757	1181
925925	7848	2104	8326	1421
1111110	9436	2823	9892	1733
1296295	10993	2922	11465	2103
1481480	12567	3338	13270	2208
1666665	14167	4055	14615	2619
1851850	15718	4150	16182	2782
2037035	17276	4555	17777	3203
2222220	18840	4959	19323	2291
2407405	20410	5374	20892	2636
2592590	21973	5805	22450	2779
2777775	23551	6241	24026	3006
2962960	25226	6602	25612	3146
3148145	26692	7028	27178	3521
3333330	28263	7437	28734	3476
3518515	29829	7850	30363	3848
3703700	31406	8236	31910	3977
3888885	32971	8629	33432	4227
4074070	34549	9044	35026	4560
4259255	36124	9621	36615	4530
4444440	37716	9884	38180	4698
4629625	39268	10263	39743	4960
4814810	40855	10847	41292	5014
5000000	42400	11121	43204	5157

Также были проведены измерения ускорения решения с помощью *OpenMP*, привязки к конкретному ядру и *GPU* по сравнению с последовательным методом, которые показывают, насколько быстрее или медленнее сработала конкретная вариация решения.

Ускорение рассчитывается как отношение времени выполнения последовательной версии алгоритма к времени выполнения параллельной версии или версии с привязкой к одному ядру. Результаты этих измерений для *Intel Core i5-1235U* приведены в таблице 5.3.

Таблица 5.3 – Замеры ускорения на *Intel Core i5-1235U*

Количество точек	Ускорение <i>OpenMP</i>	Ускорение <i>SingleCore</i>	Ускорение <i>GPU</i>
185185	4,533018868	0,844835165	2,261176471
370370	6,234962406	1,035915053	4,335947712
555555	6,653685675	1,003776752	4,608863198
740740	6,815170940	1,002199529	4,814339623
925925	7,116071429	0,996125484	5,182054616
1111110	7,028718704	0,997804725	5,338366890
1296295	7,148908858	1,002069276	5,224202627
1481480	7,209960385	0,999137323	5,200000000
1666665	7,210976838	1,000838633	4,972569444
1851850	7,330114943	0,999749169	5,722541278
2037035	7,207562680	1,000913242	5,565217391
2222220	7,300000000	1,000994400	8,290420459
2407405	7,350958126	0,998602005	8,256277401
2592590	7,223266364	0,990314985	8,258984809
2777775	6,743639922	1,012210986	8,532720198
2962960	7,234676504	0,989482263	8,575512950
3148145	7,041430777	1,009071800	8,461038961
3333330	7,277989337	1,000383850	8,514404514
3518515	7,187529748	0,998116263	8,361572536
3703700	7,063499112	0,998995164	8,347940173
3888885	6,732862903	0,999162254	8,165036675
4074070	7,288601792	0,997917320	7,994971429
4259255	6,966514460	0,998745295	7,785668722
4444440	7,168606834	1,000445458	7,957482284
4629625	7,218495014	0,999221907	7,903513996
4814810	6,529402491	0,999951712	7,801092484
5000000	7,146235843	0,997095117	7,755965293

Аналогичные измерения были проведены и на процессоре *Intel Core i5-1135G7*. Результаты приведены в таблице 5.4.

Таблица 5.4 – Замеры ускорения на *Intel Core i5-1135G7*

Количество точек	Ускорение <i>OpenMP</i>	Ускорение <i>SingleCore</i>	Ускорение <i>GPU</i>
185185	3,987551867	0,939393939	4,124463519
370370	3,835540839	0,960740946	5,463836478
555555	3,667704280	0,909318927	4,976768743
740740	3,722584469	0,929406541	5,317527519
925925	3,730038023	0,942589479	5,522871217

Продолжение таблицы 5.4

1111110	3,342543394	0,953902143	5,444893249
1296295	3,762149213	0,958831225	5,227294341
1481480	3,764829239	0,947023361	5,691576087
1666665	3,493711467	0,969346562	5,409316533
1851850	3,787469880	0,971326165	5,649892164
2037035	3,792755214	0,971817517	5,393693412
2222220	3,799153055	0,975003881	8,223483195
2407405	3,797915891	0,976928968	7,742792109
2592590	3,785185185	0,978752784	7,906801008
2777775	3,773593975	0,980229751	7,834664005
2962960	3,820963344	0,984928940	8,018436109
3148145	3,797951053	0,982117889	7,580800909
3333330	3,800322711	0,983608269	8,130897583
3518515	3,799872611	0,982412805	7,751819127
3703700	3,813258864	0,984205578	7,896907216
3888885	3,820952602	0,986210816	7,800094630
4074070	3,820101725	0,986381545	7,576535088
4259255	3,754703253	0,986590195	7,974392936
4444440	3,815864023	0,987847040	8,028097063
4629625	3,826171685	0,988048210	7,916935484
4814810	3,766479211	0,989416836	8,148185082
5000000	3,812606780	0,981390612	8,221834400

5.6 Результаты измерений с выключенным *Hyper-Threading*

Результаты измерений производительности с выключенным *Hyper-Threading* на процессорах *Intel Core i5-1225U* и *Intel Core i5-1135G7*. Ниже будут представлены аналогичные замеры, как и при включенной функции *Hyper-Threading*.

В таблице 5.5 представлены замеры времени вычисления Π методом Монте-Карло на процессоре *Intel Core i5-1225U* с выключенной функцией *Hyper-Threading*.

Таблица 5.5 – Замеры времени вычисления Π методом Монте-Карло на *Intel Core i5-1235U* с выключенным *Hyper-Threading*

Количество точек	Последовательный	Параллельный (<i>OpenMP</i>)	На одном ядре	На <i>GPU</i>
185185	1897	526	2318	793
370370	3571	682	3173	749
555555	4753	900	4765	955
740740	6410	1120	6314	1156

Продолжение таблицы 5.5

925925	7917	1400	7910	1310
1111110	9514	1643	9506	1590
1296295	11063	1994	11080	1794
1481480	12679	2268	12637	2319
1666665	14223	2519	14256	2613
1851850	15864	2796	15806	2419
2037035	17401	2983	17407	2987
2222220	18982	3310	19004	1883
2407405	20599	3548	20572	2112
2592590	22178	3839	22169	2302
2777775	23740	4043	23710	2447
2962960	25330	4399	25348	2558
3148145	26955	4653	26656	2802
3333330	28538	4790	28508	2879
3518515	30085	5183	30063	3141
3703700	31667	5460	31701	3334
3888885	33238	5586	33260	3534
4074070	34823	5766	34824	3663
4259255	36438	6047	36467	4605
4444440	37995	6208	37962	4784
4629625	39596	6596	39621	4988
4814810	41129	6662	41204	5240
5000000	42779	6973	42748	5459

Таблица 5.6 представляет замеры времени вычисления Π методом Монте-Карло на процессоре *Intel Core i5-1135G7* с выключенной функцией *Hyper-Threading*.

Таблица 5.6 – Замеры времени вычисления Π методом Монте-Карло на *Intel Core i5-1135G7* с выключенным *Hyper-Threading*

Количество точек	Последовательный	Параллельный (<i>OpenMP</i>)	На одном ядре	На <i>GPU</i>
185185	1912	487	1894	411
370370	4446	899	3148	596
555555	4715	1784	4788	956
740740	6288	1712	6283	1215
925925	7868	2146	7854	1513
1111110	9426	2580	9427	1791
1296295	10989	2987	10994	1999
1481480	12564	3390	12575	2190
1666665	14134	3822	14138	2647

Продолжение таблицы 5.6

1851850	15705	4245	15731	2851
2037035	17279	4671	17284	3036
2222220	18858	5095	18860	2189
2407405	20426	5698	20416	2496
2592590	21987	6074	22036	2689
2777775	23578	6345	23562	3035
2962960	25133	6759	25135	2970
3148145	26702	7182	26715	3555
3333330	28260	7616	28300	3538
3518515	29853	8017	29842	3803
3703700	31399	8677	31422	3989
3888885	32969	8892	32992	4183
4074070	34550	9297	34587	4463
4259255	36106	9677	36145	4646
4444440	37702	10123	37709	4876
4629625	39251	10565	39273	5238
4814810	40849	10966	40907	5120
5000000	42405	11476	42420	5291

Результаты измерения ускорения на *Intel Core i5-1225U* с выключенным *Hyper-Threading* приведены в таблице 5.7.

Таблица 5.7 – Замеры ускорения на *Intel Core i5-1225U*

Количество точек	Ускорение <i>OpenMP</i>	Ускорение <i>SingleCore</i>	Ускорение <i>GPU</i>
185185	3,606463878	0,818377912	2,392181589
370370	5,236070381	1,125433344	4,767690254
555555	5,281111111	0,997481637	4,976963351
740740	5,723214286	1,015204308	5,544982699
925925	5,655000000	1,000884956	6,043511450
1111110	5,790626902	1,000841574	5,983647799
1296295	5,548144433	0,998465704	6,166666667
1481480	5,590388007	1,003323574	5,467442863
1666665	5,646288210	0,997685185	5,443168772
1851850	5,673819742	1,003669493	6,558081852
2037035	5,833389205	0,999655311	5,825577503
2222220	5,734743202	0,998842349	10,080722250
2407405	5,805806088	1,001312464	9,753314394
2592590	5,777025267	1,000405972	9,634231103
2777775	5,871877319	1,001265289	9,701675521
2962960	5,758126847	0,999289885	9,902267396

Продолжение таблицы 5.7

3148145	5,793036750	1,011216987	9,619914347
3333330	5,957828810	1,001052336	9,912469608
3518515	5,804553347	1,000731797	9,578159822
3703700	5,799816850	0,998927479	9,498200360
3888885	5,950232725	0,999338545	9,405206565
4074070	6,039368713	0,999971284	9,506688507
4259255	6,025797916	0,999204760	7,912703583
4444440	6,120328608	1,000869290	7,942098662
4629625	6,003032141	0,999369021	7,938251804
4814810	6,173671570	0,998179788	7,849045802
5000000	6,134949089	1,000725180	7,836416926

Аналогичные измерения проведены на процессоре *Intel Core i5-1135G7*. Результаты приведены в таблице 5.8.

Таблица 5.8 – Замеры ускорения на *Intel Core i5-1135G7*

Количество точек	Ускорение <i>OpenMP</i>	Ускорение <i>SingleCore</i>	Ускорение <i>GPU</i>
185185	3,926078029	1,009503696	4,652068127
370370	4,945494994	1,412325286	7,459731544
555555	2,642937220	0,984753551	4,932008368
740740	3,672897196	1,000795798	5,175308642
925925	3,666356011	1,001782531	5,200264375
1111110	3,653488372	0,999893922	5,262981575
1296295	3,678942082	0,999545206	5,497248624
1481480	3,706194690	0,999125249	5,736986301
1666665	3,698063841	0,999717075	5,339629770
1851850	3,699646643	0,998347213	5,508593476
2037035	3,699207878	0,999710715	5,691370224
2222220	3,701275761	0,999893955	8,614892645
2407405	3,584766585	1,000489812	8,183493590
2592590	3,619855120	0,997776366	8,176645593
2777775	3,715996848	1,000679060	7,768698517
2962960	3,718449475	0,999920430	8,462289562
3148145	3,717905876	0,999513382	7,511111111
3333330	3,710609244	0,998586572	7,987563595
3518515	3,723712112	1,000368608	7,849855377
3703700	3,618646998	0,999268029	7,871396340
3888885	3,707714800	0,999302861	7,881663878
4074070	3,716252555	0,998930234	7,741429532
4259255	3,731115015	0,998921013	7,771416272

Продолжение таблицы 5.8

4444440	3,724390003	0,999814368	7,732157506
4629625	3,715191671	0,999439819	7,493508973
4814810	3,725059274	0,998582150	7,978320313
5000000	3,695102823	0,999646393	8,014553015

5.7 Анализ полученных результатов

Для сравнения разницы между выполнением задачи на обоих процессорах (при включенной гиперпоточности) с использованием *OpenMP* была найдена разницы между замерами времени на *Intel Core i5-1135G7* и *Intel Core i5-1225U* (результаты на *1135G7* минус результаты на *1225U*).

Результаты приведены в таблице 5.9.

Таблица 5.9 – Сравнение времени для метода с *OpenMP*

Количество точек	<i>Intel Core i5-1135G7</i>	<i>Intel Core i5-1225U</i>	Разница
185185	482	424	58
370370	906	532	374
555555	1285	719	566
740740	1687	936	751
925925	2104	1120	984
1111110	2823	1358	1465
1296295	2922	1558	1364
1481480	3338	1767	1571
1666665	4055	1986	2069
1851850	4150	2175	1975
2037035	4555	2433	2122
2222220	4959	2620	2339
2407405	5374	2818	2556
2592590	5805	3086	2719
2777775	6241	3577	2664
2962960	6602	3524	3078
3148145	7028	3886	3142
3333330	7437	3939	3498
3518515	7850	4202	3648
3703700	8236	4504	3732
3888885	8629	4960	3669
4074070	9044	4799	4245
4259255	9621	5256	4365
4444440	9884	5326	4558

Продолжение таблицы 5.9

4629625	10263	5515	4748
4814810	10847	6343	4504
5000000	11121	6004	5117

Аналогичные результаты для последовательного метода при включенной гиперпоточности приведены в таблице 5.10.

Таблица 5.10 – Сравнение времени для последовательного метода

Количество точек	<i>Intel Core i5-1135G7</i>	<i>Intel Core i5-1225U</i>	Разница
185185	1922	1922	0
370370	3475	3317	158
555555	4713	4784	-71
740740	6280	6379	-99
925925	7848	7970	-122
1111110	9436	9545	-109
1296295	10993	11138	-145
1481480	12567	12740	-173
1666665	14167	14321	-154
1851850	15718	15943	-225
2037035	17276	17536	-260
2222220	18840	19126	-286
2407405	20410	20715	-305
2592590	21973	22291	-318
2777775	23551	24122	-571
2962960	25226	25495	-269
3148145	26692	27363	-671
3333330	28263	28668	-405
3518515	29829	30202	-373
3703700	31406	31814	-408
3888885	32971	33395	-424
4074070	34549	34978	-429
4259255	36124	36616	-492
4444440	37716	38180	-464
4629625	39268	39810	-542
4814810	40855	41416	-561
5000000	42400	42906	-506

В таблице 5.11 приведены вычисления для метода с привязкой к одному ядру с включенной функцией *HyperThreading*.

Таблица 5.11 – Сравнение времени для метода с привязкой к одному ядру

Количество точек	<i>Intel Core i5-1135G7</i>	<i>Intel Core i5-1225U</i>	Разница
185185	2046	2275	-229
370370	3617	3202	415
555555	5183	4766	417
740740	6757	6365	392
925925	8326	8001	325
1111110	9892	9566	326
1296295	11465	11115	350
1481480	13270	12751	519
1666665	14615	14309	306
1851850	16182	15947	235
2037035	17777	17520	257
2222220	19323	19107	216
2407405	20892	20744	148
2592590	22450	22509	-59
2777775	24026	23831	195
2962960	25612	25766	-154
3148145	27178	27117	61
3333330	28734	28657	77
3518515	30363	30259	104
3703700	31910	31846	64
3888885	33432	33423	9
4074070	35026	35051	-25
4259255	36615	36662	-47
4444440	38180	38163	17
4629625	39743	39841	-98
4814810	41292	41418	-126
5000000	43204	43031	173

Результаты для вычисления на *GPU* приведены в таблице 5.12.

Таблица 5.12 – Сравнение времени для метода с использованием *GPU*

Количество точек	<i>Intel Core i5-1135G7</i>	<i>Intel Core i5-1225U</i>	Разница
185185	466	850	-384
370370	636	765	-129
555555	947	1038	-91
740740	1181	1325	-144
925925	1421	1538	-117
1111110	1733	1788	-55

Продолжение таблицы 5.12

1296295	2103	2132	-29
1481480	2208	2450	-242
1666665	2619	2880	-261
1851850	2782	2786	-4
2037035	3203	3151	52
2222220	2291	2307	-16
2407405	2636	2509	127
2592590	2779	2699	80
2777775	3006	2827	179
2962960	3146	2973	173
3148145	3521	3234	287
3333330	3476	3367	109
3518515	3848	3612	236
3703700	3977	3811	166
3888885	4227	4090	137
4074070	4560	4375	185
4259255	4530	4703	-173
4444440	4698	4798	-100
4629625	4960	5037	-77
4814810	5014	5309	-295
5000000	5157	5532	-375

Такие же расчеты с вычислением разницы были проведены для всех четырёх методов при выключенной функции гиперпоточности. Результаты этих измерений приведены в таблицах 5.13-5.16.

Таблица 5.13 – Сравнение времени для метода с *OpenMP*

Количество точек	<i>Intel Core i5-1135G7</i>	<i>Intel Core i5-1225U</i>	Разница
185185	487	526	-39
370370	899	682	217
555555	1784	900	884
740740	1712	1120	592
925925	2146	1400	746
1111110	2580	1643	937
1296295	2987	1994	993
1481480	3390	2268	1122
1666665	3822	2519	1303
1851850	4245	2796	1449
2037035	4671	2983	1688

Продолжение таблицы 5.13

2222220	5095	3310	1785
2407405	5698	3548	2150
2592590	6074	3839	2235
2777775	6345	4043	2302
2962960	6759	4399	2360
3148145	7182	4653	2529
3333330	7616	4790	2826
3518515	8017	5183	2834
3703700	8677	5460	3217
3888885	8892	5586	3306
4074070	9297	5766	3531
4259255	9677	6047	3630
4444440	10123	6208	3915
4629625	10565	6596	3969
4814810	10966	6662	4304
5000000	11476	6973	4503

Таблица 5.14 – Сравнение времени для последовательного метода

Количество точек	<i>Intel Core i5-1135G7</i>	<i>Intel Core i5-1225U</i>	Разница
185185	1912	1897	15
370370	4446	3571	875
555555	4715	4753	-38
740740	6288	6410	-122
925925	7868	7917	-49
1111110	9426	9514	-88
1296295	10989	11063	-74
1481480	12564	12679	-115
1666665	14134	14223	-89
1851850	15705	15864	-159
2037035	17279	17401	-122
2222220	18858	18982	-124
2407405	20426	20599	-173
2592590	21987	22178	-191
2777775	23578	23740	-162
2962960	25133	25330	-197
3148145	26702	26955	-253
3333330	28260	28538	-278
3518515	29853	30085	-232
3703700	31399	31667	-268
3888885	32969	33238	-269

Продолжение таблицы 5.14

4074070	34550	34823	-273
4259255	36106	36438	-332
4444440	37702	37995	-293
4629625	39251	39596	-345
4814810	40849	41129	-280
5000000	42405	42779	-374

Таблица 5.15 – Сравнение времени для метода с привязкой к одному ядру

Количество точек	<i>Intel Core i5-1135G7</i>	<i>Intel Core i5-1225U</i>	Разница
185185	1894	2318	-424
370370	3148	3173	-25
555555	4788	4765	23
740740	6283	6314	-31
925925	7854	7910	-56
1111110	9427	9506	-79
1296295	10994	11080	-86
1481480	12575	12637	-62
1666665	14138	14256	-118
1851850	15731	15806	-75
2037035	17284	17407	-123
2222220	18860	19004	-144
2407405	20416	20572	-156
2592590	22036	22169	-133
2777775	23562	23710	-148
2962960	25135	25348	-213
3148145	26715	26656	59
3333330	28300	28508	-208
3518515	29842	30063	-221
3703700	31422	31701	-279
3888885	32992	33260	-268
4074070	34587	34824	-237
4259255	36145	36467	-322
4444440	37709	37962	-253
4629625	39273	39621	-348
4814810	40907	41204	-297
5000000	42420	42748	-328

Таблица 5.16 – Сравнение времени для метода с использованием GPU

Количество точек	<i>Intel Core i5-1135G7</i>	<i>Intel Core i5-1225U</i>	Разница
185185	411	793	-382
370370	596	749	-153
555555	956	955	1
740740	1215	1156	59
925925	1513	1310	203
1111110	1791	1590	201
1296295	1999	1794	205
1481480	2190	2319	-129
1666665	2647	2613	34
1851850	2851	2419	432
2037035	3036	2987	49
2222220	2189	1883	306
2407405	2496	2112	384
2592590	2689	2302	387
2777775	3035	2447	588
2962960	2970	2558	412
3148145	3555	2802	753
3333330	3538	2879	659
3518515	3803	3141	662
3703700	3989	3334	655
3888885	4183	3534	649
4074070	4463	3663	800
4259255	4646	4605	41
4444440	4876	4784	92
4629625	5238	4988	250
4814810	5120	5240	-120
5000000	5291	5459	-168

Графики производительности процессоров представлены в приложении Д. На основе проведённого анализа данных и графиков производительности можно сделать общий вывод о том, что эффективность каждого из сравниваемых процессоров сильно зависит от типа вычислительной нагрузки и режима работы. В многопоточных сценариях, использующих технологию *OpenMP*, процессор *i5-1235U* демонстрирует существенное преимущество, обеспечивая ускорение до 7,3 раз по сравнению с последовательным выполнением. Это объясняется его более эффективной работой в условиях параллелизма и ощутимым положительным влиянием *Hyper-Threading*, которое позволяет лучше задействовать вычислительные ресурсы ядер.

График многопоточной производительности на рисунке Д.2 наглядно подтверждает лидерство *1235U*.

Однако в задачах, требующих высокой однопоточной производительности, ситуация меняется. Последовательное выполнение и работа в режиме с привязкой к одному физическому ядру оказываются сильными сторонами процессора *i5-1135G7*. Как видно из детальных графиков на рисунке Д.5 и рисунке Д.6, именно он показывает лучшее время в этих режимах. При этом интересно, что технология *Hyper-Threading*, будучи полезной для многопоточных вычислений, в однопоточном режиме может приводить к незначительным, но стабильным потерям производительности, что особенно заметно на результатах *1135G7* при работе на одном ядре.

Отдельного внимания заслуживает выполнение вычислений на графическом процессоре. Оба процессора используют интегрированную графику архитектуры *Iris Xe*, однако *i5-1135G7* стабильно демонстрирует лучшее время выполнения по сравнению с *i5-1235U*. Это преимущество может быть связано с различиями в конфигурации графических исполнительных блоков, тактовой частоте *GPU* или особенностях энергопредела и теплопакета конкретных мобильных платформ, что влияет на устойчивую производительность в продолжительных вычислениях.

Сравнивая разницу во времени выполнения между процессорами для разных методов, становится очевидна их специализация. Для многопоточного *OpenMP* разница значительна и в пользу *1235U*, в то время как для последовательного метода и работы на *GPU* разница, хоть и менее выраженная, но в пользу *1135G7*. Это подчёркивает важность соответствия архитектурных особенностей процессора целевым задачам. В заключение можно сказать, что выбор между этими процессорами должен основываться на профиле планируемой нагрузки: *i5-1235U* является более сбалансированным решением для многопоточных *CPU*-задач, в то время как *i5-1135G7* может быть оптимален для сценариев с более высокими требованиями к однопоточной производительности или с более активным использованием возможностей интегрированного графического ядра.

ЗАКЛЮЧЕНИЕ

В рамках курсового проекта было выполнено комплексное исследование, направленное на сравнение производительности двух процессоров корпорации *Intel* – *Core i5-1235U* архитектуры *Alder Lake* и *Core i5-1135G7* архитектуры *Tiger Lake*. Цель работы заключалась в проведении сравнительного анализа эффективности данных процессоров при реализации алгоритма вычисления числа Π методом Монте-Карло. Для достижения этой цели были изучены архитектурные особенности процессоров, реализована программа на языке *C++*, поддерживающая последовательные, многопоточные, *GPU*-ускоренные вычисления, а также режим работы на одном физическом ядре, проведены серии точных измерений и выполнен детальный анализ полученных результатов.

Проведённые эксперименты позволили выявить характерные особенности производительности каждого процессора в зависимости от типа вычислительной нагрузки. Процессор *Intel Core i5-1235U* продемонстрировал существенное превосходство в многопоточных сценариях с использованием технологии *OpenMP*. Максимальное ускорение параллельной реализации относительно последовательной составило до 7,3 раза, что свидетельствует об эффективности его гибридной архитектуры и удачном использовании технологии *Hyper-Threading* для распараллеливания независимых итераций алгоритма Монте-Карло.

В то же время процессор *Intel Core i5-1135G7* показал более высокую результативность в задачах, требующих высокой однопоточной производительности. При последовательном выполнении и работе в режиме с привязкой к одному физическому ядру он стабильно обеспечивал меньшее время вычислений по сравнению с *i5-1235U*. Анализ влияния *Hyper-Threading* также выявил, что в однопоточных режимах эта технология может создавать незначительные дополнительные накладные расходы, несколько снижая абсолютную производительность.

При выполнении вычислений на графическом процессоре оба тестируемых процессора, оснащённые интегрированной графикой *Iris Xe*, показали высокую эффективность, однако *i5-1135G7* обеспечил стабильно лучшее время выполнения. Это преимущество может объясняться различиями в конкретной конфигурации графических исполнительных блоков или особенностях драйверов, что подчёркивает важность учёта не только поколения архитектуры, но и специфики реализации в конкретной мобильной платформе.

Таким образом, практическая значимость выполненной работы заключается в доказательстве того, что оценка производительности процессора не может быть однозначной и сильно зависит от характера решаемых задач. *Intel Core i5-1235U*, благодаря прогрессивной гибридной архитектуре, является предпочтительным выбором для многопоточных вычислительных нагрузок. *Intel Core i5-1135G7* сохраняет конкурентоспособность и может оказаться более оптимальным решением для сценариев с более высокими требованиями к однопоточной производительности или при более активном использовании интегрированной графики.

Все задачи курсового проекта успешно решены. Разработанное программное обеспечение позволило получить воспроизводимые и статистически достоверные результаты. Проведённый анализ даёт чёткое понимание сильных сторон каждой из сравниваемых архитектур и предоставляет практические рекомендации для выбора процессора в зависимости от целевых вычислительных потребностей.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- [1] Сравнение процессоров [Электронный ресурс]. – Режим доступа: <https://technical.city/ru/cpu/Core-i5-1135G7-protiv-Core-i5-1235U>. – Дата доступа: 21.09.2025.
- [2] Core i5-1235U: технические характеристики и тесты [Электронный ресурс]. – Режим доступа: <https://technical.city/ru/cpu/Core-i5-1235U>. – Дата доступа: 24.09.2025.
- [3] Core i5-1135G7: технические характеристики и тесты [Электронный ресурс]. – Режим доступа: <https://technical.city/ru/cpu/Core-i5-1135G7>. – Дата доступа: 24.09.2025.
- [4] PassMark Software - CPU Benchmark Charts [Электронный ресурс]. – <https://www.cpubenchmark.net/>. – Дата доступа: 24.09.2025.
- [5] Geekbench 5 - Geekbench Blog [Электронный ресурс]. – <https://www.geekbench.com/blog/2019/09/geekbench-5/>. – Дата доступа: 24.09.2025.
- [6] Cinebench 2024, R23, R20, R15 - All Version [Электронный ресурс]. – Режим доступа: <https://cinebench.net/>. – Дата доступа: 24.09.2025.
- [7] Как самому за один вечер собрать минимальную ОС Linux из исходного кода / Хабр [Электронный ресурс]. – Режим доступа: <https://habr.com/ru/companies/ruvds/articles/702570/>. – Дата доступа: 24.09.2025.
- [8] Анатомия GNU/Linux [Электронный ресурс]. – Режим доступа: <https://habr.com/ru/articles/531872/>. – Дата доступа: 24.09.2025.
- [9] Понимание архитектуры операционной системы Linux [Электронный ресурс]. – Режим доступа: <https://www.youstable.com/ru/blog/architecture-of-the-linux-operating-system/>. – Дата доступа: 24.09.2025.
- [10] Структура и типы файловых систем в Linux [Электронный ресурс]. – Режим доступа: <https://timeweb.com/ru/community/articles/struktura-i-tipy-faylovyh-sistem-v-linux>. – Дата доступа: 24.09.2025.
- [11] 30 лет проекту GNU / Хабр [Электронный ресурс]. – Режим доступа: <https://habr.com/ru/articles/195478/>. – Дата доступа: 24.09.2025.
- [12] История Linux. Вкратце о главном / Хабр [Электронный ресурс]. – Режим доступа: <https://habr.com/ru/articles/95646/>. – Дата доступа: 24.09.2025.
- [13] Linux kernel version history [Электронный ресурс]. – Режим доступа: https://en.wikipedia.org/wiki/Linux_kernel_version_history#Releases_5.x.y. – Дата доступа: 24.09.2025.
- [14] Что стоит знать задумавшемуся о переходе на Ubuntu / Хабр [Электронный ресурс]. – Режим доступа: <https://habr.com/ru/articles/106165/>. – Дата доступа: 24.09.2025.
- [15] Директивы OpenMP [Электронный ресурс]. – Режим доступа: <https://learn.microsoft.com/ru-ru/cpp/parallel/openmp/reference/openmpdirectives?view=msvc-170>. – Дата доступа: 25.10.2025.

[16] Microsoft: Processes and threads [Электронный ресурс]. – Режим доступа: <https://learn.microsoft.com/ru-ru/windows/win32/procthread/processes-and-threads>. – Дата доступа: 27.10.2025.

[17] CPU Affinity [Электронный ресурс]. – Режим доступа: <https://www.cryptoprofi.info/?p=6848>. – Дата доступа: 27.10.2025.

ПРИЛОЖЕНИЕ А

(обязательное)

Справка о проверке на заимствования

☐ Название ▾	Статус ▾	Дата загрузки ▾	Оригинальность
☐ .TXT ВВЕДЕНИЕ		07 Дек 2025 21:28	98,73%
☐ .TXT 1 АРХИТЕКТУРА ВЫЧИСЛИТЕЛЬНОЙ СИСТЕМЫ		07 Дек 2025 21:18	100%
☐ .TXT 2 ПЛАТФОРМА ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ		07 Дек 2025 21:03	100%
☐ .TXT 2 ПЛАТФОРМА ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ		07 Дек 2025 20:47	100%
☐ .TXT 3 ТЕОРЕТИЧЕСКОЕ ОБОСНОВАНИЕ РАЗРАБОТКИ ПРОГРАММНОГО		07 Дек 2025 20:37	100%
☐ .TXT 3 ТЕОРЕТИЧЕСКОЕ ОБОСНОВАНИЕ РАЗРАБОТКИ ПРОГРАММНОГО		07 Дек 2025 20:16	100%
☐ .TXT ЗАКЛЮЧЕНИЕ		07 Дек 2025 20:08	98,02%
☐ .TXT 5 СРАВНЕНИЕ ПРОИЗВОДИТЕЛЬНОСТИ ПРОЦЕССОРОВ. Часть 2		07 Дек 2025 19:55	100%
☐ .TXT 5 СРАВНЕНИЕ ПРОИЗВОДИТЕЛЬНОСТИ ПРОЦЕССОРОВ. Часть 1		07 Дек 2025 19:40	100%
☐ .TXT 4 ПРОЕКТИРОВАНИЕ ФУНКЦИОНАЛЬНЫХ ВОЗМОЖНОСТЕЙ ПРОГ		07 Дек 2025 19:33	100%
☐ .TXT 4 ПРОЕКТИРОВАНИЕ ФУНКЦИОНАЛЬНЫХ ВОЗМОЖНОСТЕЙ ПРОГ		07 Дек 2025 19:23	100%

Рисунок А.1 – Справка о проверке на заимствования

ПРИЛОЖЕНИЕ Б

(обязательное)

Листинг программного кода

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <chrono>
#include <random>
#include <omp.h>
#include <cstring>
#include <iomanip>
#include <fstream>
#include <hwloc.h>
#include <numeric>
#include <cmath>
#include <limits>
#include <sstream>

#define CL_HPP_TARGET_OPENCL_VERSION 200
#define CL_HPP_MINIMUM_OPENCL_VERSION 200
#include <CL/opencl.hpp>

#ifdef M_PI
    #define M_PI 3.141592653589793238462643383279502884L
#endif

hwloc_topology_t topology;

struct PiResults
{
    long double calculation_time;
    long double pi_value;
    long double relative_error;
    int threads_used;
};

class GPUPiCalculator
{
private:
    cl::Context context;
    cl::CommandQueue queue;
    cl::Program program;
    cl::Kernel monte_carlo_kernel;
    bool available;

public:
    GPUPiCalculator() : available(false)
    {
        try
        {
            std::vector<cl::Platform> platforms;
            cl::Platform::get(&platforms);
            if (platforms.empty())
            {
                std::cerr << "OpenCL платформы не найдены" << std::endl;
                return;
            }

            cl::Platform selected_platform;
```

```

    cl::Device selected_device;
    bool found_gpu = false;

    for (auto &platform : platforms)
    {
        std::vector<cl::Device> devices;
        platform.getDevices(CL_DEVICE_TYPE_GPU, &devices);

        for (auto &device : devices)
        {
            std::string device_name =
device.getInfo<CL_DEVICE_NAME>();
            std::string platform_name =
platform.getInfo<CL_PLATFORM_NAME>();

            std::cout << "Найдено GPU устройство: " << device_name
                << " на платформе: " << platform_name <<
std::endl;

            // Предпочтение Intel iGPU
            if (platform_name.find("Intel") != std::string::npos)
            {
                selected_platform = platform;
                selected_device = device;
                found_gpu = true;
                std::cout << "Выбрана Intel iGPU" << std::endl;
                break;
            }
            else if (!found_gpu)
            {
                selected_platform = platform;
                selected_device = device;
                found_gpu = true;
                std::cout << "Выбрана GPU: " << device_name <<
std::endl;
            }
        }
        if (found_gpu)
            break;
    }

    if (!found_gpu)
    {
        std::cerr << "GPU устройства не найдены" << std::endl;
        return;
    }

    context = cl::Context(selected_device);
    queue = cl::CommandQueue(context, selected_device);

    std::string kernel_source = R"(
        uint lcg(uint* seed) {
            *seed = *seed * 1103515245u + 12345u;
            return *seed;
        }

        float random_float(uint* seed) {
            uint r = lcg(seed);
            return (float)r / 4294967296.0f;
        }

        __kernel void monte_carlo_pi(__global int* results,

```

```

                                int points_count,
                                uint seed_base) {
    int tid = get_global_id(0);
    if (tid >= points_count) return;

    // Генерируем уникальный seed для каждого потока
    uint local_seed = seed_base + (uint)tid * 2654435761u;
    if (local_seed == 0) local_seed = 1;

    float x = random_float(&local_seed);
    float y = random_float(&local_seed);

    if (x*x + y*y <= 1.0f) {
        results[tid] = 1;
    } else {
        results[tid] = 0;
    }
}

)";

cl::Program::Sources sources;
sources.push_back({kernel_source.c_str(),
kernel_source.length()});

program = cl::Program(context, sources);
try {
    program.build();
} catch (const std::exception& e) {
    std::string build_log =
program.getBuildInfo<CL_PROGRAM_BUILD_LOG>(selected_device);
    std::cerr << "Ошибка компиляции OpenCL kernel: " << e.what()
<< std::endl;

    std::cerr << "Build log:" << std::endl;
    std::cerr << build_log << std::endl;
    throw;
}

monte_carlo_kernel = cl::Kernel(program, "monte_carlo_pi");

available = true;
std::cout << "OpenCL инициализирован успешно" << std::endl;
}
catch (const std::exception &e)
{
    std::cerr << "Ошибка инициализации GPU: " << e.what() <<
std::endl;
    available = false;
}

}

bool isAvailable() const { return available; }

PiResults calculate_pi_gpu_timed(long long points_count)
{
    if (!available)
    {
        std::cerr << "GPU недоступен" << std::endl;
        return {0, 0, 0, 0};
    }

    PiResults results = {0, 0, 0, 0};

```

```

std::random_device rd;
std::mt19937 gen(rd());
std::uniform_int_distribution<unsigned int> seed_dist;
unsigned int seed = seed_dist(gen);
if (seed == 0) seed = 1;

cl::Buffer results_buf(context, CL_MEM_WRITE_ONLY,
                        points_count * sizeof(int));

auto start = std::chrono::high_resolution_clock::now();

monte_carlo_kernel.setArg(0, results_buf);
monte_carlo_kernel.setArg(1, static_cast<int>(points_count));
monte_carlo_kernel.setArg(2, seed);

try {
    queue.enqueueNDRangeKernel(monte_carlo_kernel, cl::NullRange,
                                cl::NDRange(points_count),
cl::NullRange);
    queue.finish();
} catch (const std::exception& e) {
    std::cerr << "Ошибка выполнения OpenCL kernel: " << e.what() <<
std::endl;
    return {0, 0, 0, 0};
}

std::vector<int> gpu_results;
gpu_results.resize(points_count);
queue.enqueueReadBuffer(results_buf, CL_TRUE, 0,
                        points_count * sizeof(int),
                        gpu_results.data());

auto end = std::chrono::high_resolution_clock::now();

int points_inside = std::accumulate(gpu_results.begin(),
gpu_results.end(), 0);

double pi_estimate = 4.0 * static_cast<double>(points_inside) /
static_cast<double>(points_count);
double relative_error = std::abs(pi_estimate - M_PI) / M_PI * 100.0;

results.calculation_time =
std::chrono::duration_cast<std::chrono::microseconds>(
    end - start).count();
results.pi_value = pi_estimate;
results.relative_error = relative_error;
results.threads_used = -1; // GPU не использует потоки CPU

return results;
}
};

GPUPiCalculator g_gpu_calculator;

// Инициализация hwloc
void initTopology()
{
    hwloc_topology_init(&topology);
    hwloc_topology_load(topology);
}

// Освобождение ресурсов hwloc

```

```

void cleanupTopology()
{
    hwloc_topology_destroy(topology);
}

// Установка аффинити к конкретному физическому ядру
void setAffinityToPhysicalCore(int physicalCoreId)
{
    int depth = hwloc_get_type_or_below_depth(topology, HWLOC_OBJ_CORE);
    hwloc_obj_t core = hwloc_get_obj_by_depth(topology, depth,
physicalCoreId);

    if (core == NULL)
    {
        std::cerr << "Ошибка: физическое ядро " << physicalCoreId << " не
найдено" << std::endl;
        return;
    }

    if (hwloc_set_cpubind(topology, core->cpuset, HWLOC_CPUBIND_THREAD) != 0)
    {
        std::cerr << "Ошибка установки аффинити к физическому ядру " <<
physicalCoreId << std::endl;
        char *str;
        hwloc_bitmap_asprintf(&str, core->cpuset);
        std::cerr << "CPUSet: " << str << std::endl;
        free(str);
    }
}

PiResults calculate_pi_sequential_timed(long long points_count)
{
    std::random_device rd;
    std::mt19937 gen(rd());
    std::uniform_real_distribution<float> dist(0.0f, 1.0f);

    auto start = std::chrono::high_resolution_clock::now();

    long long points_inside = 0;
    for (long long i = 0; i < points_count; ++i)
    {
        float x = dist(gen);
        float y = dist(gen);

        if (x * x + y * y <= 1.0f)
        {
            points_inside++;
        }
    }

    double pi_estimate = 4.0 * static_cast<double>(points_inside) /
static_cast<double>(points_count);
    auto end = std::chrono::high_resolution_clock::now();

    long double time_us =
std::chrono::duration_cast<std::chrono::microseconds>(
    end - start).count();
    long double relative_error = std::abs(pi_estimate - M_PI) / M_PI * 100.0;

    return {time_us, pi_estimate, relative_error, 1};
}

```

```

PiResults calculate_pi_openmp_timed(long long points_count, int num_threads =
-1)
{
    if (num_threads > 0) {
        omp_set_num_threads(num_threads);
    }

    long long points_inside = 0;

    auto start = std::chrono::high_resolution_clock::now();

    #pragma omp parallel reduction(+ : points_inside)
    {
        std::random_device rd;
        std::mt19937 gen(rd() + omp_get_thread_num());
        std::uniform_real_distribution<float> dist(0.0f, 1.0f);

        #pragma omp for
        for (long long i = 0; i < points_count; ++i)
        {
            float x = dist(gen);
            float y = dist(gen);

            if (x * x + y * y <= 1.0f)
            {
                points_inside++;
            }
        }
    }

    double pi_estimate = 4.0 * static_cast<double>(points_inside) /
static_cast<double>(points_count);
    auto end = std::chrono::high_resolution_clock::now();

    long double time_us =
std::chrono::duration_cast<std::chrono::microseconds>(
    end - start).count();
    long double relative_error = std::abs(pi_estimate - M_PI) / M_PI * 100.0;

    int actual_threads = (num_threads > 0) ? num_threads :
omp_get_max_threads();
    return {time_us, pi_estimate, relative_error, actual_threads};
}

PiResults calculate_pi_openmp_timed_wrapper(long long points_count) {
    return calculate_pi_openmp_timed(points_count);
}

PiResults calculate_pi_gpu_timed(long long points_count)
{
    return g_gpu_calculator.calculate_pi_gpu_timed(points_count);
}

PiResults calculate_pi_single_core_timed(long long points_count, int coreId)
{
    setAffinityToPhysicalCore(coreId);
    return calculate_pi_sequential_timed(points_count);
}

// Шаблонная функция для тестирования
template <typename TimeUnits>
PiResults TestPiTimed(PiResults (*pi_func)(long long),

```

```

        long long points_count,
        int iterations)
{
    long double time_sum = 0;
    long double pi_sum = 0;
    long double error_sum = 0;
    int threads = 0;

    for (int i = 0; i < iterations; i++)
    {
        auto results = pi_func(points_count);
        time_sum += results.calculation_time;
        pi_sum += results.pi_value;
        error_sum += results.relative_error;
        threads = results.threads_used;
    }

    return {time_sum / iterations, pi_sum / iterations, error_sum /
iterations, threads};
}

PiResults TestSingleCoreTimed(PiResults (*pi_func)(long long, int),
        long long points_count,
        int iterations, int coreId)
{
    long double time_sum = 0;
    long double pi_sum = 0;
    long double error_sum = 0;

    for (int i = 0; i < iterations; i++)
    {
        setAffinityToPhysicalCore(coreId);
        auto results = pi_func(points_count, coreId);
        time_sum += results.calculation_time;
        pi_sum += results.pi_value;
        error_sum += results.relative_error;
    }

    return {time_sum / iterations, pi_sum / iterations, error_sum /
iterations, 1};
}

void printTopologyInfo()
{
    int depth = hwloc_get_type_or_below_depth(topology, HWLOC_OBJ_CORE);
    int num_cores = hwloc_get_nbobjs_by_depth(topology, depth);

    std::cout <<
    "=====\n";
    std::cout << "ИНФОРМАЦИЯ О ТОПОЛОГИИ ПРОЦЕССОРА\n";
    std::cout <<
    "=====\n";
    std::cout << "Обнаружено физических ядер: " << num_cores << std::endl;

    for (int i = 0; i < num_cores; i++)
    {
        hwloc_obj_t core = hwloc_get_obj_by_depth(topology, depth, i);
        char *cpuset_str;
        hwloc_bitmap_asprintf(&cpuset_str, core->cpuset);
    }
}

```

```

        std::cout << "Физическое ядро " << i << ": логические CPU " <<
cpuset_str << std::endl;
        free(cpuset_str);
    }
    std::cout <<
"=====
====\n\n";
}

// Вывод результатов
void printResultsTable(long long points_count, int iterations,
                        const PiResults &seq_results, const PiResults
&omp_results,
                        const PiResults &single_results, const PiResults
&gpu_results)
{
    const int COL_METHOD = 12;
    const int COL_THREADS = 8;
    const int COL_TIME = 12;
    const int COL_PI = 14;
    const int COL_ERROR = 12;

    const int TOTAL_WIDTH = COL_METHOD + COL_THREADS + COL_TIME + COL_PI +
COL_ERROR + 14;

    std::cout << std::string(TOTAL_WIDTH, '=') << "\n";
    std::cout << "СРАВНЕНИЕ ПРОИЗВОДИТЕЛЬНОСТИ ВЫЧИСЛЕНИЯ ПИ МЕТОДОМ МОНТЕ-
КАРЛО\n";
    std::cout << std::string(TOTAL_WIDTH, '=') << "\n";
    std::cout << "Количество точек: " << points_count << std::endl;
    std::cout << "Итерации: " << iterations << std::endl;
    std::cout << "GPU доступен: " << (g_gpu_calculator.isAvailable() ? "Да" :
"Нет") << "\n\n";

    std::cout << std::string(TOTAL_WIDTH, '=') << "\n";
    std::cout << "ТАБЛИЦА ПРОИЗВОДИТЕЛЬНОСТИ (время в микросекундах)\n";
    std::cout << std::string(TOTAL_WIDTH, '=') << "\n";

    // Заголовок таблицы
    std::cout << std::string(TOTAL_WIDTH, '-') << "\n";
    std::cout << "| " << std::left << std::setw(COL_METHOD) << "    Метод    "
        << " | " << std::setw(COL_THREADS) << " Поток    "
        << " | " << std::setw(COL_TIME) << " Время(мкс) "
        << " | " << std::setw(COL_PI) << " ПИ значение "
        << " | " << std::setw(COL_ERROR) << " Ошибка(%) " << " | \n";
    std::cout << std::string(TOTAL_WIDTH, '-') << "\n";

    auto formatNumber = [](long double number) -> std::string {
        std::stringstream ss;
        ss << std::fixed << std::setprecision(8) << number;
        return ss.str();
    };

    auto formatError = [](long double error) -> std::string {
        std::stringstream ss;
        ss << std::fixed << std::setprecision(4) << error;
        return ss.str();
    };

    auto formatTime = [](long double time) -> std::string {
        std::stringstream ss;
        ss << std::fixed << std::setprecision(0) << time;

```

```

        return ss.str();
    };

    auto printRow = [&](const std::string& method, const std::string&
threads,
                    const std::string& time, const std::string& pi,
                    const std::string& error) {
        std::cout << " | " << std::left << std::setw(COL_METHOD) << method
        << " | " << std::right << std::setw(COL_THREADS) << threads
        << " | " << std::setw(COL_TIME) << time
        << " | " << std::setw(COL_PI) << pi
        << " | " << std::setw(COL_ERROR) << error << " |\n";
    };

    printRow("Последоват ",
            std::to_string(seq_results.threads_used),
            formatTime(seq_results.calculation_time),
            formatNumber(seq_results.pi_value),
            formatError(seq_results.relative_error));

    printRow("OpenMP",
            std::to_string(omp_results.threads_used),
            formatTime(omp_results.calculation_time),
            formatNumber(omp_results.pi_value),
            formatError(omp_results.relative_error));

    printRow("Одно ядро ",
            std::to_string(single_results.threads_used),
            formatTime(single_results.calculation_time),
            formatNumber(single_results.pi_value),
            formatError(single_results.relative_error));

    if (g_gpu_calculator.isAvailable()) {
        printRow("GPU",
                "-",
                formatTime(gpu_results.calculation_time),
                formatNumber(gpu_results.pi_value),
                formatError(gpu_results.relative_error));
    }

    std::cout << std::string(TOTAL_WIDTH, '-') << "\n\n";
}

void saveResultsToFile(const std::vector<std::tuple<long long, int,
PiResults, PiResults,
                    PiResults, PiResults>> &results)
{
    std::ofstream file("pi_monte_carlo_results.csv");
    if (!file.is_open())
    {
        std::cerr << "Ошибка открытия файла для записи!" << std::endl;
        return;
    }

    file <<
    "Точки,Итерации,Метод,Потоки,Время_мкс,Пи_значение,Ошибка_проценты\n";

    for (const auto &result : results)
    {
        long long points_count;
        int iterations;
        PiResults seq, omp, single, gpu;

```

```

        std::tie(points_count, iterations, seq, omp, single, gpu) = result;

        file << points_count << "," << iterations << ",Sequential," <<
seq.threads_used << ","
        << std::fixed << std::setprecision(0) << seq.calculation_time <<
", "
        << std::fixed << std::setprecision(10) << seq.pi_value << ","
        << std::fixed << std::setprecision(6) << seq.relative_error <<
"\n";

        file << points_count << "," << iterations << ",OpenMP," <<
omp.threads_used << ","
        << std::fixed << std::setprecision(0) << omp.calculation_time <<
", "
        << std::fixed << std::setprecision(10) << omp.pi_value << ","
        << std::fixed << std::setprecision(6) << omp.relative_error <<
"\n";

        file << points_count << "," << iterations << ",SingleCore," <<
single.threads_used << ","
        << std::fixed << std::setprecision(0) << single.calculation_time
<< ", "
        << std::fixed << std::setprecision(10) << single.pi_value << ","
        << std::fixed << std::setprecision(6) << single.relative_error
<< "\n";

        if (g_gpu_calculator.isAvailable()) {
            file << points_count << "," << iterations << ",GPU,0,"
            << std::fixed << std::setprecision(0) <<
gpu.calculation_time << ","
            << std::fixed << std::setprecision(10) << gpu.pi_value <<
", "
            << std::fixed << std::setprecision(6) << gpu.relative_error
<< "\n";
        }
    }
    file.close();
}

// Функции для пользовательского интерфейса
int getIntegerInput(const std::string& prompt, int min_val, int max_val) {
    int value;
    while (true) {
        std::cout << prompt;
        std::cin >> value;
        if (std::cin.fail() || value < min_val || value > max_val) {
            std::cin.clear();
            std::cin.ignore(std::numeric_limits<std::streamsize>::max(),
'\n');
            std::cout << "Ошибка! Введите целое число от " << min_val << " до
" << max_val << std::endl;
        } else {
            std::cin.ignore(std::numeric_limits<std::streamsize>::max(),
'\n');
            return value;
        }
    }
}

long long getLongLongInput(const std::string& prompt, long long min_val, long
long max_val) {
    long long value;

```

```

        while (true) {
            std::cout << prompt;
            std::cin >> value;
            if (std::cin.fail() || value < min_val || value > max_val) {
                std::cin.clear();
                std::cin.ignore(std::numeric_limits<std::streamsize>::max(),
'\n');
                std::cout << "Ошибка! Введите число от " << min_val << " до " <<
max_val << std::endl;
            } else {
                std::cin.ignore(std::numeric_limits<std::streamsize>::max(),
'\n');
                return value;
            }
        }
    }

bool getYesNoInput(const std::string& prompt) {
    char choice;
    while (true) {
        std::cout << prompt << " (y/n): ";
        std::cin >> choice;
        std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');

        if (choice == 'y' || choice == 'Y') return true;
        if (choice == 'n' || choice == 'N') return false;
        std::cout << "Ошибка! Введите 'y' или 'n'" << std::endl;
    }
}

void showMainMenu() {
    std::cout << "\n===== \n";
    std::cout << "    БЕНЧМАРК ВЫЧИСЛЕНИЯ ПИ МЕТОДОМ МОНТЕ-КАРЛО \n";
    std::cout << "===== \n";
    std::cout << "1. Автоматический тест (рекомендуемые параметры) \n";
    std::cout << "2. Ручная настройка теста \n";
    std::cout << "3. Информация о системе \n";
    std::cout << "4. Выход \n";
    std::cout << "===== \n";
}

void runAutomaticTest() {
    std::cout << "\n=== АВТОМАТИЧЕСКИЙ ТЕСТ === \n";

    // Равномерное распределение с учетом точки (0, 0) на графике
    std::vector<long long> test_sizes;
    long long start = 0;
    long long end = 5000000;
    int count = 27;
    long long step = (end - start) / count;

    for (int i = 1; i <= count; ++i) {
        test_sizes.push_back(start + i * step);
    }
    test_sizes[count - 1] = end;

    int iterations = 10;

    std::vector<std::tuple<long long, int, PiResults, PiResults,
        PiResults, PiResults>> results;

    for (long long points_count : test_sizes) {

```

```

std::cout << "Тестирование с " << points_count << " точками...\n";

auto seq_results = TestPiTimed<std::chrono::microseconds>(
    calculate_pi_sequential_timed, points_count, iterations);

auto omp_results = TestPiTimed<std::chrono::microseconds>(
    calculate_pi_omp_timed_wrapper, points_count, iterations);

auto single_results = TestSingleCoreTimed(
    calculate_pi_single_core_timed, points_count, iterations, 0);

auto gpu_results = TestPiTimed<std::chrono::microseconds>(
    calculate_pi_gpu_timed, points_count, iterations);

results.emplace_back(points_count, iterations, seq_results,
omp_results,
                    single_results, gpu_results);

    printResultsTable(points_count, iterations, seq_results, omp_results,
                    single_results, gpu_results);
}

saveResultsToFile(results);
std::cout << "Результаты сохранены в pi_monte_carlo_results.csv" <<
std::endl;
}

void runManualTest() {
    std::cout << "\n=== РУЧНАЯ НАСТРОЙКА ТЕСТА ===\n";

    // Настройка количества точек
    std::cout << "\n--- Настройка количества точек ---\n";
    std::cout << "1. Использовать предустановленные значения\n";
    std::cout << "2. Ввести вручную\n";
    int choice = getIntegerInput("Выберите вариант: ", 1, 2);

    std::vector<long long> test_sizes;
    if (choice == 1) {
        test_sizes = {
            100000,
            1000000,
            5000000,
            10000000,
            25000000,
            50000000,
            100000000,
            250000000,
            500000000
        };
    } else {
        int num_sizes = getIntegerInput("Сколько различных размеров
тестировать: ", 1, 10);
        for (int i = 0; i < num_sizes; ++i) {
            long long points = getLongLongInput(
                "Введите количество точек для теста " + std::to_string(i+1) +
": ",
                1000, 1000000000);
            test_sizes.push_back(points);
        }
    }

    // Настройка количества итераций

```

```

int iterations = getIntegerInput(
    "Введите количество итераций для усреднения (1-50): ", 1, 50);

// Выбор методов тестирования
std::cout << "\n--- Выбор методов тестирования ---\n";
bool test_sequential = getYesNoInput("Тестировать последовательную
версию?");
bool test_openmp = getYesNoInput("Тестировать OpenMP версию?");
bool test_single_core = getYesNoInput("Тестировать версию с одним
ядром?");
bool test_gpu = g_gpu_calculator.isAvailable() ?
    getYesNoInput("Тестировать GPU версию?") : false;

// Запуск тестов
std::vector<std::tuple<long long, int, PiResults, PiResults,
    PiResults, PiResults>> results;

for (long long points_count : test_sizes) {
    std::cout << "\nТестирование с " << points_count << " точками...\n";

    PiResults seq_results = {0, 0, 0, 0};
    PiResults omp_results = {0, 0, 0, 0};
    PiResults single_results = {0, 0, 0, 0};
    PiResults gpu_results = {0, 0, 0, 0};

    if (test_sequential) {
        seq_results = TestPiTimed<std::chrono::microseconds>(
            calculate_pi_sequential_timed, points_count, iterations);
    }

    if (test_openmp) {
        omp_results = TestPiTimed<std::chrono::microseconds>(
            calculate_pi_openmp_timed_wrapper, points_count, iterations);
    }

    if (test_single_core) {
        single_results = TestSingleCoreTimed(
            calculate_pi_single_core_timed, points_count, iterations, 0);
    }

    if (test_gpu) {
        gpu_results = TestPiTimed<std::chrono::microseconds>(
            calculate_pi_gpu_timed, points_count, iterations);
    }

    results.emplace_back(points_count, iterations, seq_results,
omp_results,
                        single_results, gpu_results);

    printResultsTable(points_count, iterations, seq_results, omp_results,
                        single_results, gpu_results);
}

if (getYesNoInput("Сохранить результаты в файл?")) {
    saveResultsToFile(results);
    std::cout << "Результаты сохранены в pi_monte_carlo_results.csv" <<
std::endl;
}
}

int main(int argc, char* argv[])
{

```

```

// Инициализация системы
initTopology();

std::cout << "Инициализация системы..." << std::endl;
printTopologyInfo();

int max_threads = omp_get_max_threads();
omp_set_num_threads(max_threads);
std::cout << "Максимальное количество потоков OpenMP: " << max_threads <<
std::endl;

if (g_gpu_calculator.isAvailable()) {
    std::cout << "GPU поддержка: ДОСТУПНА" << std::endl;
} else {
    std::cout << "GPU поддержка: НЕДОСТУПНА" << std::endl;
}

// Главное меню
int choice;
do {
    showMainMenu();
    choice = getIntegerInput("Выберите действие: ", 1, 4);

    switch (choice) {
        case 1:
            runAutomaticTest();
            break;
        case 2:
            runManualTest();
            break;
        case 3:
            printTopologyInfo();
            break;
        case 4:
            std::cout << "Выход из программы..." << std::endl;
            break;
    }
} while (choice != 4);
cleanupTopology();

return 0;
}

```

ПРИЛОЖЕНИЕ В
(обязательное)
Функциональная схема алгоритма,
реализующего программное средство

ПРИЛОЖЕНИЕ Г
(обязательное)
Блок схема алгоритма,
реализующего программное средство

ПРИЛОЖЕНИЕ Д
(обязательное)
Графики производительности процессоров

ПРИЛОЖЕНИЕ Е
(обязательное)
Графический интерфейс пользователя

ПРИЛОЖЕНИЕ Ж
(обязательное)
Ведомость курсового проекта